



POLITECHNIKA WARSZAWSKA
WYDZIAŁ MATEMATYKI I NAUK INFORMACYJNYCH



PRACA DYPLOMOWA INŻYNIERSKA
NA KIERUNKU INFORMATYKA

**PROJEKT E-PODRĘCZNIKA DO NAUCZANIA
PODSTAWOWEGO
E-LEARNING PROJECT FOR PRIMARY LEVEL
EDUCATION**

AUTOR:
MARTA JANKOWSKA

PROMOTOR:
DR INŻ. MARCIN LUCKNER

WARSZAWA, KWIECIEŃ 2012

.....
podpis promotora

.....
podpis autora

Streszczenie

Tematem niniejszej pracy jest utworzenie projektowej wersji e-podręcznika do nauczania podstawowego. Zakres materiału, który uwzględnia podręcznik to dodawanie i odejmowanie liczb do 10, relacje równości, mniejszości i większości oraz znajomość liter (danego alfabetu). W wersji projektowej dane mogą być wprowadzane poprzez kreślenie urządzeniem wskazującym lub z gotowych przykładów. Głównym odbiorcą danego projektu są dzieci w wieku szkolnym (przedszkolnym) oraz ich rodzice (nadzorujący pracę systemu).

Na rozwiązanie problemu tego projektu składają się dwa podstawowe zagadnienia: przetwarzanie obrazu oraz sztuczne sieci neuronowe. Metody przetwarzania obrazu (ang. *image processing*) wykorzystane są do pobierania oraz obróbki wstępnej (ang. *preprocessing*) obrazu danego symbolu (liter, cyfry, znaku). Następnie, wprowadzony obraz poddany jest działaniu sztucznej sieci neuronowej (ang. *Artificial Neural Network*), która to rozpoznaje obraz danego symbolu i zwraca rozpoznany symbol (już jako znak, a nie obraz).

W pracy wydzielić można dwie części - część studialną oraz część projektową.

Część studialna przedstawia teoretyczny zarys podejmowanych tematów (przetwarzanie obrazu oraz sztuczne sieci neuronowe).

Na początku omówiony jest problem przetwarzania obrazu, z naciskiem na obróbkę wstępną oraz szczególny przypadek rozpoznawania obrazu - rozpoznawanie obrazu pisma ręcznego.

W kolejnym rozdziale opisane zostały sztuczne sieci neuronowe - poczynając od zarysu historycznego, poprzez biologiczne podstawy, budowę i sposób działania, aż do algorytmów nauki oraz rodzajów sieci neuronowych.

Szerzej omówione zostały teoretyczne podstawy oraz algorytm nauki sieci neuronowej uczonej metodą wstecznej propagacji błędów ze współczynnikiem momentum (gdzież właśnie taka sieć została wykorzystana w danym projekcie). Dla zainteresowanych podane zostały dalsze odniesienia do literatury.

W części projektowej opisana jest aplikacja użyta do tworzenia e-podręcznika. Pokazane jest praktyczne podejście do tematu przetwarzania obrazu (pobieranie obrazu, obróbka wstępna) oraz sztucznych sieci neuronowych (wykorzystanie sieci do rozpoznawania pobranego już obrazu).

W pracy znaleźć można szczegółowy opis projektu, dokumentację kodu oraz raport z przeprowadzonych testów.

Jak już wspomniano wcześniej, głównym problemem było stworzenie modułu przetwarzania obrazu oraz jego rozpoznawania (sztuczne sieci neuronowe). Dla wygody użytkownika stworzony został graficzny interfejs użytkownika. Aby system działał poprawnie, należało odpowiednio ze sobą połączyć te 3 komponenty (np. zadanie przetwarzania obrazu polegało na doprowadzeniu go do postaci wektorowej, która następnie może być wejściem sieci neuronowej). Szczegółowy opis wykorzystanych komponentów oraz ich adaptacji na potrzeby projektu znajduje się w rozdziale poświęconym modułom projektu.

Raport z przeprowadzonych testów przedstawia wyniki działania sieci neuronowej. Niestety, pomimo, iż sieć bardzo dobrze sprawdza się w rozpoznawaniu cyfr oraz znaków, nie radzi sobie najlepiej z rozpoznawaniem liter. Możliwe przyczyny niesprawności sieci opisane są w podsumowaniu.

Abstract

The topic of this dissertation is the creation of writing and mathematics learning software for primary-level education. The tasks for children covered in this software include basic addition and subtraction (up to 10), greater-than, less-than, and equality relationships, as well as a platform to practice correctly writing numbers and the letters of the alphabet. Data can be entered either by hand-writing numbers and letters into an input device or by selecting finished examples from a table. The intended users of this project are kindergarden-aged children and their parents who will act to supervise the system (neural network training).

The solution to the problem of this project consists of two basic areas: image processing and artificial neural networks. Image processing methods are used to collect and preprocess the image of a symbol (letter, number, or character). Next, the preprocessed image is used as an input of an artificial neural network, which recognizes the image of the symbol and returns the recognized symbol (as a character, not an image).

This work can be separated into two parts - study and design.

The study part presents the theoretical framework of the themes (image processing and artificial neural networks). First, the problem of image processing is dealt with, with emphasis on preprocessing in the special case of handwriting image recognition. The next chapter describes artificial neural networks, beginning with an historical overview by examining the biological basis, construction, and operation of biological neural networks in order to understand the types of algorithms needed to create artificial ones. The theoretical scientific basis of neural networks is discussed in some detail as well as a backpropagation algorithm (exactly this type of network has been implemented in this project). The focus also includes, among other things, the activation function, different models of artificial neurons and various learning algorithms (used for comparison). References to literature are given for those interested who wish to further educate themselves.

In the project thus described is an application used to create an e-book virtual learning platform. This platform is a practical approach to the topic of image processing (including image enhancement and preprocessing) and artificial neural networks (used for character recognition).

Reading through this work, you will find a detailed description of the design and documentation code, as well as reports on tests carried out.

As previously mentioned, the main problem was to create an image processing module and then to apply character recognition using artificial neural networks. For the convenience of the user (a child and his or her parent), a graphical user interface was created.

The included report presents the results of tests of the network neurons. Unfortunately, despite the fact that the network works very well in identifying numbers and special characters ($=$, $<$, $>$), it has difficulty in the recognition of letters. Possible inefficiencies responsible for this are described in the summary.

Spis treści

Streszczenie	3
Abstract	5
Wstęp	9
1. Wprowadzenie	11
1.1. Geneza problemu	11
1.2. Cel pracy	12
1.3. Zakres pracy	12
2. Rozpoznawanie obrazów	13
2.1. Metody rozpoznawania obrazów	13
2.1.1. Formy danych obrazowych	13
2.1.2. Zadanie rozpoznawania obrazów	14
2.2. Przetwarzanie obrazu pisma	18
2.2.1. Segmentacja	19
2.3. Zastosowania	20
3. Sztuczne sieci neuronowe	21
3.1. Zarys historyczny	21
3.2. Wprowadzenie, podstawy biologiczne	21
3.3. Budowa i działanie sztucznego neuronu	22
3.3.1. Funkcja aktywacji	24
3.4. Modele sztucznych neuronów	26
3.4.1. Perceptron	26
3.4.2. Model Adeline	27
3.4.3. Model neuronu sigmoidalnego	27
3.4.4. Model neuronu Hebba	28
3.5. Struktura i podstawy działania sieci	30
3.6. Uczenie sieci	31
3.6.1. Algorytm wstecznej propagacji błędów	31
3.6.2. Algorytm wstecznej propagacji błędów ze współczynnikiem momentum	33
3.6.3. Algorytm zmiennej metryki	34
3.6.4. Algorytm Levenberga- Marquardta	34
3.6.5. Rekurencyjna metoda najmniejszych kwadratów	34
3.7. Rodzaje sieci	34
3.8. Dobór architektury sieci	36
4. Opis projektu	37

4.1.	Inspiracje	37
4.2.	Założenia projektu	37
4.3.	Analiza biznesowa	37
4.3.1.	Słownik	37
4.3.2.	Use case	38
4.3.3.	Specyfikacja uzupełniająca	38
4.3.4.	Zatwierdzone wymagania	39
4.4.	Wybór modelu wytwórczego	40
4.5.	Instrukcja użytkownika	41
4.5.1.	Instrukcja dla rodzica	41
4.5.2.	Instrukcja dla dziecka	43
4.6.	Opis modułów systemu	52
4.6.1.	Dokładny opis modułów systemu	53
5.	Dokumentacja kodu	59
5.1.	Pliki	59
5.2.	Dokładny opis plików aplikacji	59
5.2.1.	<i>dzialania.m / dzialania.fig</i>	59
5.2.2.	<i>budowa_sieci.m / budowa_sieci.fig</i>	63
5.2.3.	<i>my_train.m</i>	64
5.2.4.	<i>my_test.m</i>	65
5.2.5.	<i>my_check.m</i>	66
5.2.6.	<i>statystyka_sieci.m</i>	66
5.2.7.	<i>preprocess.m</i>	67
5.2.8.	<i>wektor_uczacy.m</i>	68
5.2.9.	<i>generuj_dodawanie.m</i>	68
5.2.10.	<i>generuj_odejmowanie.m</i>	68
5.2.11.	<i>generuj_porownianie.m</i>	69
5.2.12.	<i>generuj_slowo.m</i>	70
6.	Raport z przeprowadzanych testów	71
6.1.	Testy jednostkowe	71
6.2.	Testy integracyjne	71
6.3.	Testy akceptacyjne	71
6.4.	Testy sieci neuronowej	71
6.5.	Opis zmian w stosunku do poprzednio dostarczonych dokumentów	77
	Podsumowanie	79
	Spis rysunków	81
	Bibliografia	83

Wstęp

Inteligencja, zdolność samodzielnego uczenia się jest tym, co pozwoliło ludzkości osiągnąć dzisiejszy poziom technologiczny.

Sztuczna inteligencja jest dziedziną nauki, która, naśladując mechanizm działania człowieka (na różnych płaszczyznach), próbuje osiągnąć cele nie do zrealizowania przy tradycyjnym, algorytmicznym podejściu (do danych problemów).

Jedną z ciekawszych gałęzi sztucznej inteligencji, która opiera swą strukturę i działanie na żywych organizmach są sztuczne sieci neuronowe.

W tej pracy przybliżę zagadnienie sztucznych sieci neuronowych, ich budowę, sposób działania oraz zastosowanie. Wykorzystam sztuczne sieci neuronowe do rozpoznawania pisma ręcznego. Rozpoznawanie obrazów jest także jedną z ważniejszych dziedzin dzisiejszej informatyki. Przy tak wielkiej ilości napływających informacji, dobrze jest móc wyszukać te właściwe.

Tworzony przeze mnie projekt wykorzystuje sztuczne sieci neuronowe do rozpoznawania pisma ręcznego. Program ma służyć dzieciom do nauki pisania (liter, cyfr, znaków) oraz może być pomocny w liczeniu.

Rozdział 1

Wprowadzenie

1.1. Geneza problemu

W procesie rozpoznawania obrazu, a w szczególności rozpoznawania pisma istnieje już niemalże niezawodny system. Jest to ludzki mózg. Obszar odpowiedzialny za czynność widzenia składa się ze 100 miliardów neuronów [1], a każdy neuron z przynajmniej tysiąca synaps. Tak wielka sieć neuronowa złożona jest z wielu mniejszych sieci, które rozwijały się przez tysiące lat. Sieci te działają równolegle, tworząc wielki system rozpoznawania wzorców. System ten ma pewną przewagę nad systemem komputerowym.

Człowiek może się domyśleć znaczenia wyrazów (nawet jeśli kolejność liter nie jest zachowana), natomiast system komputerowy wychwyci nawet najmniejszy błąd.

Cołwziek ptsozrgea wiśat ianzecj żin
kopmuetr dlatgeo zcasami obiketwyna
oneca metrwej mazysny jset lpezsa.

Rysunek 1.1: Ludzki mózg potrafi przewidzieć kolejność liter.

W pewnym sensie jest to przewaga systemu komputerowego. Nie dopuszcza on bowiem nawet najmniejszych błędów.

Ludzki organizm wykazuje jednak pewne podobieństwa do systemu komputerowego stworzonego do rozpoznawania wzorców. Przypadkowe zniszczenia lub wady pewnych regionów kory mózgowej osłabiają zdolności rozpoznawania wzrokowego. Często ludzie z uszkodzeniami kory mózgowej odpowiadającej za proces widzenia, potrafią rozpoznawać poszczególne litery, niestety nie potrafią rozpoznać całego słowa, rozpoznają poszczególne obiekty, lecz nie potrafią uchwycić całości [7].

Wiele systemów komputerowych działa podobnie. Potrafią one rozpoznawać poszczególne obiekty (np. litery), lecz nie potrafią złożyć danych składników w sensowną całość.

Najnowsze modele sieci neuronowych wykorzystują reakcje oraz interakcje pomiędzy systemami tak, jak dzieje się to w korze mózgowej i dzięki temu zapewniają odpowiedni poziom przetwarzania danych.

1.2. Cel pracy

Celem tej pracy jest stworzenie programu edukacyjnego, który byłby pomocny w nauce pisanie.

W części pisemnej pracy, nacisk położony jest na problem rozpoznawania obrazu oraz sztucznych sieci neuronowych. Powstały projekt bazuje bowiem na zagadnieniach z tych dziedzin. Aby pismo ręczne mogło zostać rozpoznane przez komputer (sieć neuronową) musi najpierw zostać odpowiednio przekształcone (pozyskiwanie obrazu, rozpoznawanie).

1.3. Zakres pracy

W części pisemnej pracy opisane jest zagadnienie rozpoznawania obrazów oraz sztucznych sieci neuronowych.

Najpierw przytoczone zostały pojęcia związane z metodami rozpoznawania obrazu. Opisane zostały kolejne etapy przetwarzania obrazu, a w szczególności obrazu pisma. Przytoczone zostały różne podejścia do tego tematu. Na końcu podano zastosowania rozpoznawania obrazu (nie tylko rozpoznawanie pisma). Największy nacisk położony został na rozdział poświęcony sieciom neuronowym.

Przedstawiono biologiczne podstawy działania sieci oraz ewolucję od prostego perceptronu po neurony sigmoidalne. Przedstawiona została architektura oraz różne rodzaje sieci. Uwaga skupiona została na algorytmie uczenia (algorytm wstecznej propagacji błędów) oraz na doborze architektury sieci.

Ostatnia część pracy dotyczy projektu programistycznego. Opisana jest funkcjonalność, instrukcje dla użytkownika oraz opis techniczny programu.

Rozdział 2

Rozpoznawanie obrazów

2.1. Metody rozpoznawania obrazów

Metody rozpoznawania obrazów są metodami tworzenia opisu obrazu wejściowego lub klasyfikacji obrazu do danej (szczególnej) klasy. W pewnym sensie proces ten jest odwrotny do grafiki komputerowej - zaczyna się od obrazu a kończy się abstrakcyjnym zbiorem opisującym ten obraz. Przetwarzanie obrazów zajmuje się znajdowaniem konturów. Równoważnym działaniem w grafice komputerowej jest wypełnianie konturów. Operacje te są odwrotne, lecz cechują je pewne wspólne zagadnienia.

2.1.1. Formy danych obrazowych

Rozważając przetwarzanie obrazów można podzielić je na cztery klasy (zaproponowane w [8]) odnoszące się bardziej do sposobu ich przedstawiania i przetwarzania aniżeli do rzeczywistej formy wizualnej.

- Obrazy o pełnej gradacji kontrastów i kolorowe (klasa I)
- Obrazy dwupoziomowe lub „kilkukolorowe”(klasa II)
- Krzywe ciągłe i linie proste (klasa III)
- Punkty lub wieloboki (klasa IV)

Klasa I - Obrazy o pełnej gradacji kontrastów i kolorowe

Obrazy te są dokładnym przedstawieniem rzeczywistości. Ich reprezentacją są macierze z elementami całkowitymi nazywanymi *element obrazu (pixel)*.

Do reprezentowania obrazów kolorowych mogą służyć trzy macierze lub jedna macierz, w której różne bity każdego elementu odpowiadają poszczególnym kolorom (RGB).

Klasa II - Obrazy dwupoziomowe lub „kilkukolorowe”

Przykładem takiego obrazu może być strona tekstu.

Reprezentacją obrazów tej klasy mogą być macierze z jednym bitem na element, a także „mapy” ponieważ zawierają one jednoznacznie określone obszary jednego koloru.

Klasa III - Krzywe ciągłe i linie proste

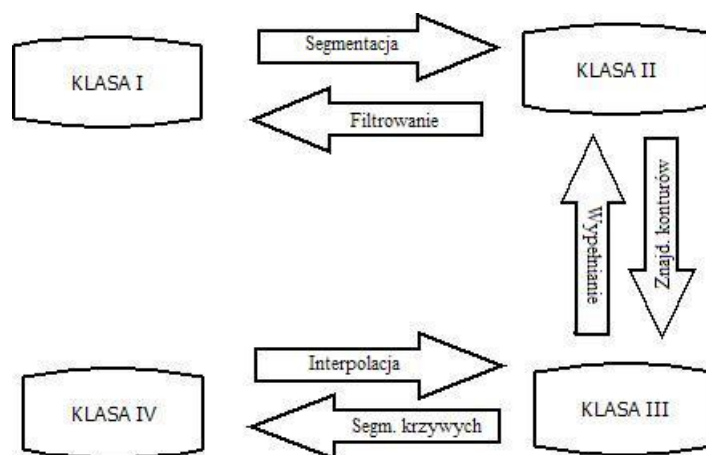
Przykładem takich obrazów mogą być kontury obszarów lub wykresów.

Dane te są ciągami punktów, które mogą być reprezentowane jako ich współrzędne -jest to jednak metoda nieefektywna.

Klasa IV - Punkty lub wieloboki

Obrazy tej klasy to najczęściej zbiory oddzielnych punktów, które są tak od siebie oddalone, iż trudno byłoby reprezentować je przez kod łańcuchowy. W zamian stosowane są tablice ich współrzędnych.

Obrazy tego typu najczesciej stosowane są w grafice komputerowej. Pomimo iż same obrazy należą do klasy II lub I, to ich wewnętrzna reprezentacja należy do klasy IV.



Rysunek 2.1: Transformacje pomiędzy klasami. (Na podstawie [8], s.62)

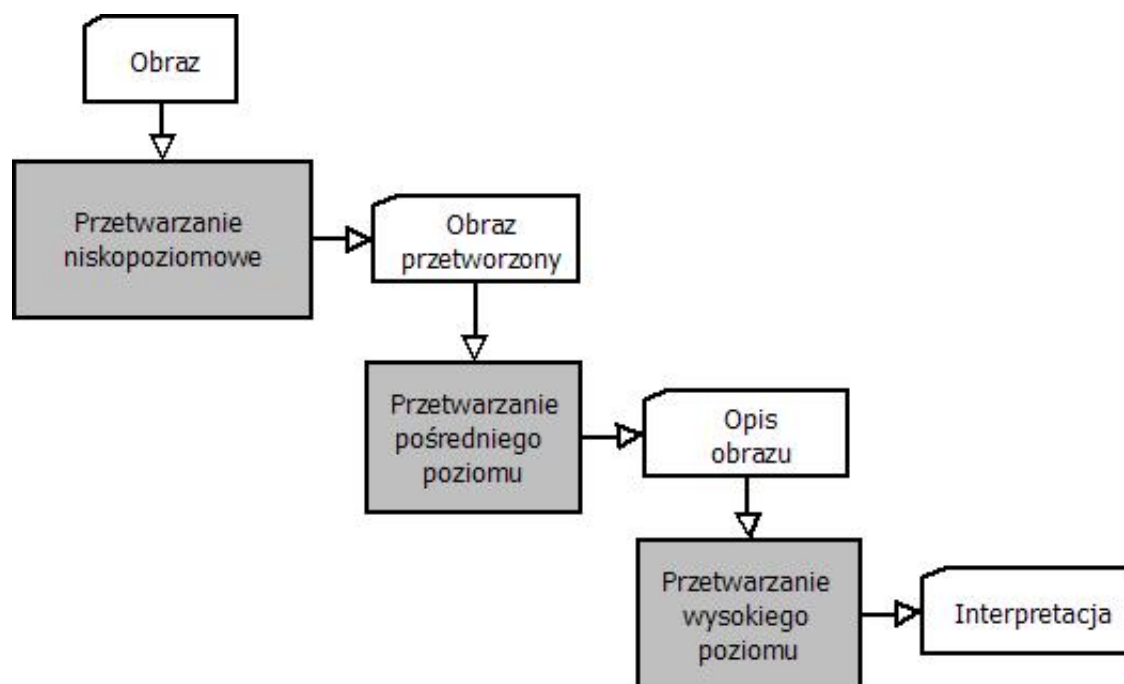
2.1.2. Zadanie rozpoznawania obrazów

Proces rozpoznawania obrazów składa się z kilku czynności. Aby obraz analogowy mógł zostać przetwarzany komputerowo należy zamienić go najpierw na tablicę liczb.

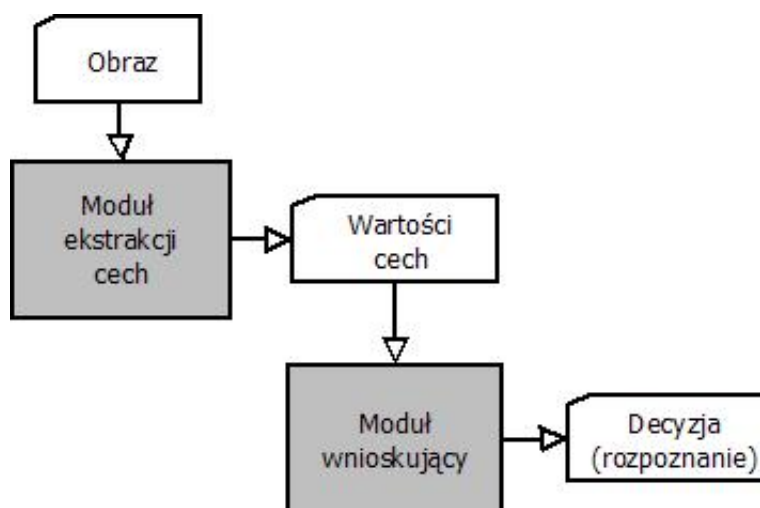
Ogólny schemat rozpoznawania obrazu przedstawia się następująco:

- Pozyskiwanie (rejestracja, akwizycja)
- Przetwarzanie (często segmentacja)
- Analiza
- Interpretacja

Często również spotkać można inne podejścia do tematu rozpoznawania obrazu (Rysunek 2.2 oraz 2.3).



Rysunek 2.2: Etapy rozpoznawania obrazów I.



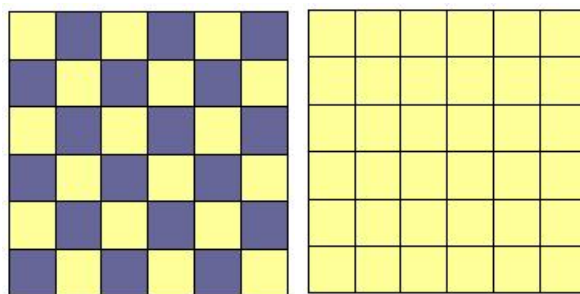
Rysunek 2.3: Etapy rozpoznawania obrazów II.

Pierwszym etapem jest *akwizycja*, czyli inaczej pozyskiwanie obrazów.

W etapie tym pożądanym jest, aby pozyskany obraz dyskretny był topologicznie równoważny oryginalnemu obrazowi ciągłemu. Przy zbyt niskiej częstotliwości próbkowania przestrzennego może dojść do utraty informacji.

Do pozyskiwania obrazu potrzebne są urządzenia takie jak:

- kamera (kolorowa lub stopnie szarosci)



Rysunek 2.4: Obraz rzeczywisty oraz obraz zarejestrowany

- stereoskopia
- kamery typu vidicon (fotoprzewodnictwo), elementy CCD(charge-coupled devices)
- skaner
- digitizer, („tablet”)
- dalmierz laserowy lub ultradźwiękowy
- oddziaływanie fizyczne z otoczeniem (robotyka)
- aparatura diagnostyczna (np. tomografy)

Pobrany obraz poddany zostaje przetwarzaniu wstępnemu [7]. Do tych czynności należą:

- dyskretyzacja / digitalizacja (ang. *digitization*)
- binaryzacja (ang. *binarization*)
- eliminacja szumu (ang. *noise elimination*)
- szkielek (ang. *Skeleton -thinning, skeletonizing*)
- normalizacja (ang. *normalizing*).

Proces dyskretyzacji składa się z:

- Próbkowania - wyboru zbioru punktów z pola obserwacji, pomiaru parametrów obrazu i stosowania tego w analizie całego obrazu.
- Kwantowania - opisu danych pomiarowych za pomocą skończonej liczby cyfr.

Rozdzielczość przestrzenna dotyczy gęstości punktów próbkowania.

Rozdzielczość poziomów jasności lub koloru dotyczy wierności przedstawienia danych pomiarowych.

Jednym z procesów zaliczanych do przetwarzania wstępnego jest tworzenie tzn. *szkieletu*. Celem tej operacji jest redukcja kształtu obiektu do grafu.

Z definicji punkt szkieletowy (ang. *skeleton*) zbioru (obszaru) R to punkt, który ma więcej niż jednego najbliższego sąsiada na brzegu B zbioru R . Szkielet R to zbiór wszystkich punktów szkieletowych R .

Inaczej punkty szkieletowe to punkty o tej własności, że są środkami okręgów całkowicie zawartych w R , przy czym nie ma innego okręgu o tym samym środku i większym promieniu zawartego całkowicie w R .

Szkielet nadaje się głównie do analizy cienkich obiektów. Wykazuje on podatność na drobne

zniekształcenia brzegów.

Kolejny etap to ekstrakcja cech. Składa się na niego segmentacja (jawna lub niejawna) oraz wykrywanie cech głównych.

Segmentacja obrazu to identyfikowanie obszarów obrazu, których wygląd jest jednolity dla obserwatora. Identyfikowanie obrazu oznacza jednoznaczne ustalenie, do jakiego obszaru (lub innych elementów np. krawędzi) należy dany punkt obrazu. Jednolitość może zostać zdefiniowana na bardzo różne sposoby (poziom jasności, charakterystyka kolorystyczna, tekstura, itd.).

Segmentacja jest dość ważnym zagadnieniem w rozpoznawaniu obrazów. Jedną z cech tego zagadnienia jest przejście od ilościowej do jakościowej informacji (reprezentacji obrazu).

Segmentacja jest również bardzo często silnie zależna od zastosowania. Implikuje to konieczność uwzględnienia wiedzy dziedzinowej (w stopniu znacznie wyższym od etapów wcześniej omawianych).

Następnym etapem jest klasyfikacja. Polega ona na rozpoznawaniu wzorców (np. rozpoznawanie liter za pomocą sztucznych sieci neuronowych) oraz weryfikacji w kontekście przy użyciu podstawowej wiedzy (np. sprawdzenie otrzymanego słowa za pomocą słownika).

Ostatnim z etapów jest interpretacja otrzymanego obrazu.

W literaturze spotkać można również inny podział etapów rozpoznawania obrazów. Przykładem jest poniższy diagram.



Rysunek 2.5: Etapy rozpoznawania obrazów III.

2.2. Przetwarzanie obrazu pisma

Rozpoznawanie pisma (a w szczególności pisma ręcznego) jest szczególnym przypadkiem rozpoznawania obrazów.

Wyróżnia się dwa typy rozpoznawania pisma: *online* (czyli rozpoznawanie obrazu pisma w trakcie procesu pisania) oraz *offline* (rozpoznawanie obrazu pisma już wcześniej pobranego) [10].

W niniejszej pracy zajmę się pobieraniem oraz rozpoznawaniem pisma w trybie *offline*.

Architektura typowego systemu komputerowego do rozpoznawania pisma składa się z następujących modułów:

- przetwarzanie wstępne (ang. *preprocessing*)
- wydzielanie cech (ang. *feature extraction*)
- klasyfikacja (ang. *classification*), czyli moduł rozpoznający (ang. *recognizer*).

Przy pobieraniu (akwizycji) obrazu pisma oraz pomiarze cech wykonywanych jest szereg czynności i związanych z nimi błędów pomiarowych [2].

Proces pobierania obrazu pisma wygląda tak samo jak proces pobierania innych obrazów. W procesie tym natępuje zmiana analogowej postaci obrazu do postaci cyfrowej (za pomocą aparatu fotograficznego, skanera lub innych urządzeń). Aby przenieść obraz pisma do postaci elektronicznej, dokonywane jest pierwsze przekształcenie - digitalizacja (dyskretyzacja). Jak już zostało wspomniane wyżej, proces ten polega na przekształceniu ciągłego obrazu (sygnału analogowego) w obraz cyfrowy. Składa się on z próbkowania oraz kwantyzacji (kwantowania). Urządzenia dokonujące przekształceń obrazów do postaci cyfrowej wyposażone są w przetwornik optyczno-elektryczny. Wartość sygnału z tego przetwornika może być próbkowana w różnych odstępach (im mniejsze odstęp tym wierniejsza reprezentacja obrazu). Rozdzielczość danego urządzenia podawana jest przeważnie w jednostkach DPI (ang. *dot per inch*), czyli liczbie punktów obrazu na jednostkę długości jednego cala. Błąd próbkowania określony jest następującym wzorem:

$$\varepsilon_p[mm] = \frac{1[dot] \cdot 25,4[mm]}{1["] \cdot rozdzielczość[dpi]}. \quad (2.1)$$

Przykładowo, obraz zeskanowany z rozdzielczością 600 dpi obarczony jest błędem próbkowania $\varepsilon_p = 0,04mm$ (wielkość ta jest równa jednemu pikselowi obrazu).

Kwantyzacja polega na zmianie ciągłego zbioru wartości pewnej wielkości na dyskretny podzbiór tych wartości. Wartość sygnału poddana próbkowaniu jest zatem najbliższą liczbą ze zbioru wartości z dyskretnego podzbioru wartości wielkości mierzonej. Kwantyzacja obrazu pisma polega na zmianie pełnej gamy odcieni szarości (lub składowych kolorowych) obrazu na wartości dyskretnie. Przeważnie zbiór ciągły zostaje podzielony na pewną liczbę (N_{kol}) równych przedziałów (ΔN_{kol}) wartości funkcji. Błąd kwantyzacji ε_k przedstawia się następująco:

$$\varepsilon_k = \pm \frac{\Delta N_{kol}}{2}. \quad (2.2)$$

Najczęściej spotykana jest szesnastobitowa reprezentacja odcieni szarości dająca liczbę przedziałów $N_{kol} = 65.536$. Często spotykana jest również dwudziestoczerobitowa reprezentacja, dająca $N_{kol} = 16.777.216$ (ponad 16 milionów odcieni lub też kolorów).

Przekształcenia te prowadzą do uzyskania wartości w postaci cyfrowej.

Obraz pisma przeważnie zmieniany jest z postaci kolorowej lub odcieni szarości na obraz czarno-biały. Proces ten nazywany jest *binaryzacją*. Polega on na zmianie wartości dyskretnych sygnałów na dwie grupy o wartościach 0 dla koloru podłoża (tła) oraz 1 dla koloru materiału kryjącego. Często popełniany jest tutaj błąd pierwszego rodzaju, polegający na zakwalifikowaniu obszaru liter jako punkty tła oraz błąd drugiego rodzaju polegający na zakwalifikowaniu punktu tła jako obrazu liter. Wartość tego błędu zależy od jakości obrazu linii pisma.

Istnieje wiele algorytmów binaryzacji. Najczęściej ustalana jest dana wartość progowa. Jeśli dany kolor jest poniżej tej wartości, klasyfikowany jest np. jako tło, powyżej - jako obraz. Algorytmy binaryzacji wnoszą również dodatkową niepewność do pomiaru.

Przekształcanie wstępne składa się także z wielu innych etapów (niż te wspomniane wcześniej). Jednym z nich jest np. odsumowanie obrazu. Powstają wtedy kolejne błędy pomiarowe, wynikające z zabierania lub dodawania do obrazu pisma pojedynczych pikseli (lub - w przypadku zdefiniowania wartości progowej - grupy pikseli).

2.2.1. Segmentacja

Segmentacja obrazu pisma (tak samo jak w ogólnym przypadku) oznacza podział obrazu na jednorodne (homogeniczne) pod względem danej własności obszary. Przykładem segmentacji może być binaryzacja. Jest to segmentacja pod względem jasności punktów obrazu. Segmentacja na wyrazy, litery i gramy (części składowe liter) odnosi się nie tylko do własności obrazu (np. kolor czy jasność), lecz także do właściwości treściowych czyli semantycznych. Trudno jest bowiem dokonać poprawnej segmentacji wyrazów nie znając treści tekstu pisma.

W systemie percepcyjnym człowieka segmentacja obrazu przebiega łącznie ze zrozumieniem treści danego obrazu.

Obrazy pisma zostały przez człowieka ręcznie posegmentowane na wyrazy i litery. Podział wyrazów lub zespołów znaków na poszczególne znaki związany jest z systemem wiązań. Gdy poszczególne wyrazy, litery ani nawet gramy nie są powiązane, segmentacja nie jest szczególnie trudnym problemem. W przypadku wiązań pomiędzy tymi elementami często, aby przeprowadzić właściwą segmentację stosowane są różne algorytmy podziału wyrazu na litery. W systemie *online* można śledzić przebieg linii pisma. Niestety, w systemach *offline* informacja ta nie jest dostępna. Możliwe jest rekonstruowanie z danych *offline* przebiegu linii pisma. Stosowane są również metody oparte na analizie histogramów rozkładów częstości występowania w rzutach poziomych i pionowych ciągów punktów w kolorach podłoża pisarskiego i materiału kryjącego o poszczególnych długościach. Zagadnienia te wykraczają jednak poza ramy tej pracy.

2.3. Zastosowania

Rozpoznawanie obrazu znajduje szereg zastosowań w dzisiejszym świecie. Najczęściej spotykane zastosowania to:

- polepszanie jakości obrazu dla potrzeb dalszej analizy przez człowieka (ang. *image enhancement*, *image restoration*).
- analiza obrazu (ang. *image analysis*, *pattern recognition*) - ekstrakcja innej informacji z obrazu (niekoniecznie przypominającej cechy używane przez człowieka)
- widzenie komputerowe (ang. *computer vision*) - prowadzi do interpretacji sceny, silny nacisk na 3D, w szczególności:
widzenie maszynowe (ang. *machine vision*) - zorientowane na zastosowania inżynierskie
- rozumienie obrazu (ang. *image understanding*) - konieczny udział reprezentacji symbolicznej

Rozdział 3

Sztuczne sieci neuronowe

3.1. Zarys historyczny

Prace nad poznaniem procesów zachodzących w mózgu, które mają wpływ na inteligentny sposób jego działania, prowadzone były już od bardzo dawna przez wielu badaczy z całego świata. Jednak prawdziwym początkiem dziedziny sztucznych sieci neuronowych było przedstawienie matematycznego modelu neuronu przez *McCulloch*'a i *Pitts*'a w 1943 roku [9]. Model ten pozwalał na budowę dowolnie skomplikowanych urządzeń cyfrowych (bez możliwości uczenia się).

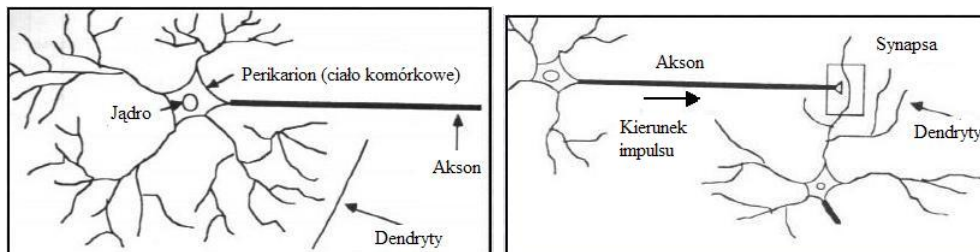
W 1949 roku *Donald Hebb* odkrył, że informacja może być przechowywana jako wartość wag pomiędzy połączeniami poszczególnych neuronów i zaproponował pierwszy algorytm uczenia takiej sieci. Polegał on na zmianach wag połączeń - nazwany on został regułą *Hebba*. W 1958 roku *Rosenblatt* stworzył model perceptronu (neurony z połączeniami jednokierunkowymi). *Rosenblatt* dowiódł zbieżności algorytmu uczenia w 1962 roku. Kojelnymi modelami były ADELIN (ang. Adaptive Linear Neuron) oraz MADELINE (multiple-Adeline). Był to pierwszy komercyjny mikrokomputer, powstał na Uniwersytecie Stanforda, a autorami projektu byli *Widrow* i *Hoff*. W 1969 roku *Minsky* i *Papert* wykazali ograniczenia jednowarstwowych sieci typu PERCEPTRON (zadanie XOR). Ich odkrycie zahamowało początkowy gwałtowny wzrost zainteresowania i postęp w tej dziedzinie. Pomimo tego powstawało wiele nowych pomysłów i modeli sieci. Można tu wymienić sieć Cerebellatron *Dawida Mara* (sterowanie robotem), Brain in the Box Jamesa Andersona (1977 r.) działająca jak pamięć asocjacyjna [9].

Dopiero publikacja algorytmu uczenia wielowarstwowej nieliniowej sieci neuronowej przełamała zastój w badaniach nad siecią. Metodę wstecznej propagacji błędów opisał po raz pierwszy *Werbos* w 1974 roku, jednak jej popularyzacja nastąpiła dopiero w 1986 roku dzięki *Rumelhart*'owi. Umożliwiło to konstruowanie wielowarstwowych nieliniowych sieci neuronowych oraz ich skuteczne uczenie, spowodowało powrót zainteresowania sieciami neuronowymi i dalszy szybki ich rozwój [9].

3.2. Wprowadzenie, podstawy biologiczne

Jak już wspomniano wyżej, model sztucznych sieci neuronowych wzorowany jest na naturalnym systemie nerwowym, szczególnie mózgu.

Mózg złożony jest z około 10 miliardów elementarnych komórek nerwowych (neuronów). Neurony tworzą bardzo skomplikowane sieci powiązane między sobą, pracujące równolegle. Pojedynczy neuron składa się z następujących części: dendrytu, perikarionu, aksonu [2][9].



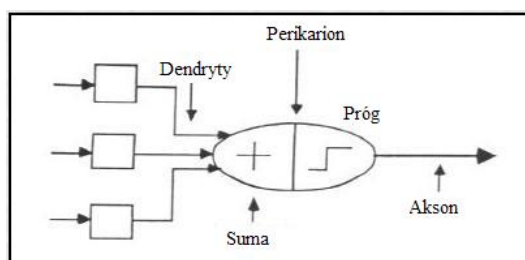
Rysunek 3.1: Budowa neuronu. (Na podstawie [9] 2.1)

Dendryty stanowią wejścia neuronu, do których prowadzony jest sygnał od innych neuronów. Perikarion jest częścią scalającą dla sygnałów z dendrytów. Funkcje wyjścia sygnału od neuronu do innych neuronów pełni akson. Odbywa się to poprzez telodendron - rozgałęzioną strukturę transportującą sygnały wyjściowe. Nośnikiem pamięci w naturalnej sieci neuronów jest synapsa - biochemiczne złącze, modyfikujące sygnały.

W sieciach naturalnych wykorzystywane jest więc chemiczno-elektryczne przekazywanie sygnałów. W komórce nerwowej generowany jest sygnał elektryczny, który zamieniany jest na sygnał chemiczny w końcu aksonu. Informacja chemiczna przekazywana jest przez neurotransmitery, których jest około 50 rodzajów. Neurony są powolne, generują sygnał z częstotliwością około kilkuset Hz.

3.3. Budowa i działanie sztucznego neuronu

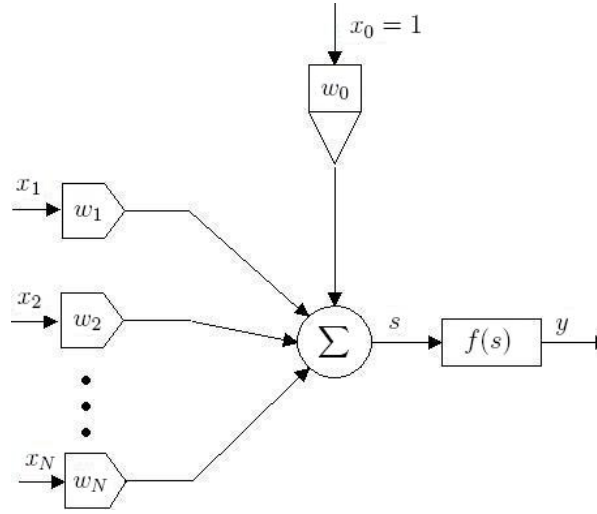
Na podstawie budowy biologicznej powstał model sztucznego neuronu oraz sieci neuronowej. Jest on znacznie uproszczony w stosunku do systemu biologicznego.



Rysunek 3.2: Schemat budowy sztucznego neuronu na podstawie neuronu biologicznego. (Na podstawie [9] 2.2)

Podobnie jak w naturalnych sieciach, sztuczne sieci neuronowe składają się z elementarnych komórek - neuronów. Sztuczny neuron ma wiele wejść i tylko jedno wyjście. Do wejść neuronów doprowadzane są sygnały wychodzące z neuronów warstwy poprzedniej (lub sygnały wejściowe). Każdy sygnał mnożony jest przez odpowiadającą mu wartość zwaną wagą (jest to efektywność synapsy). Wpływa ona na percepcję danego sygnału. Waga może być pobudzająca - dodatnia lub opóźniająca - ujemna. Jeżeli nie ma połączenia między neuronami to waga jest równa 0. Zsumowane iloczyny sygnałów i wag stanowią argument funkcji aktywacji

(Rozdział 3.3.1). Wynik funkcji jest wprowadzany na wyjście neuronu.



Rysunek 3.3: Schemat budowy sztucznego neuronu. (Na podstawie [1] 6.2.1, s.168)

Na tej podstawie, działanie pojedynczego neuronu można opisać następującymi wzorami:

$$y = f(s) \quad (3.1)$$

gdzie $f(s)$ jest *funkcją aktywacji* danego neuronu (może przyjmować różną postać, w zależności od konkretnego modelu neuronu) [1] 6.2.1, s.167.

$$s = \sum_{j=0}^N x_j w_j \quad (3.2)$$

więc:

$$y = f\left(\sum_{j=0}^N x_j w_j\right). \quad (3.3)$$

gdzie x_j jest j -tym wejściem, a w_j jest daną wagą przyporządkowaną temu wejściu (wagą synaptyczną).

N jest liczbą wejść w neuronie.

Wektor sygnałów wejściowych możemy zapisać następująco $x = [x_1, \dots, x_N]^T$

Wektor wag synaptycznych przedstawia się następująco $w = [w_0, \dots, w_N]^T$

Sygnały wejściowe $x_0, x_1, \dots, x_N$ są pomnożone przez odpowiadające im wagi $w_0, w_1, \dots, w_N$. Otrzymane wartości są następnie sumowane w wyniku czego powstaje sygnał s odpowiadający działaniu liniowej części neuronu. Następnie, otrzymany sygnał poddawany jest działaniu funkcji aktywacji (przeważnie nieliniowej). Zakłada się, iż wartość sygnału $x_0 = 1$ a wagę w_0 nazywa się *wartością progową* / *progiem* (ang. *bias*).

Często również połączenia między neuronami reprezentowane są jako *funkcje bazowe* $u(w, x)$, gdzie w jest macierzą wag, a x jest wektorem wejść. Funkcje bazowe przyjmują najczęściej jedną z dwóch form:

- Liniowe funkcje bazowe (LBF - Linear-Basis Function)
Są to funkcje bazowe pierwszego rzędu. Wartość wyjściowa sieci (neuronu) obliczana jest jako liniowa kombinacja wejść

$$u_j(w, x) = \sum_{i=1}^n w_{ij} x_i \quad (3.4)$$

- Radialne funkcje bazowe (RBF - Radial-Basis Function)
Są to funkcje bazowe drugiego rzędu. Wartość wyjściowa sieci (neuronu) reprezentuje dystans od rozważanego wzorca

$$u_j(w, x) = \sqrt{\sum_{i=1}^n (x_i - w_{ij})^2} \quad (3.5)$$

3.3.1. Funkcja aktywacji

Wartość wyjścia sieci (neuronu) określona przez funkcję bazową, $u(w, x)$ jest przekształcana za pomocą *funkcji aktywacji* neuronu.

Wybór funkcji aktywacji zależy od problemu jaki stawiamy sieci do rozwiązania. Dla sieci wielowarstwowych najczęściej stosowane są funkcje nieliniowe, ponieważ neurony o takich charakterystykach wykazują największe zdolności do nauki. Zdolności te polegają na możliwości odwzorowania w sposób płynny dowolnej zależności pomiędzy wejściem, a wyjściem sieci. Umożliwia to otrzymanie na wyjściu sieci informacji ciągłej.

Wymagane cechy funkcji aktywacji to:

- Ciągłe przejście pomiędzy swoją wartością maksymalną i minimalną
- Łatwa do obliczenia ciągła pochodna
- Możliwość wprowadzenia do argumentu parametru do ustalania krzywej.

Wyjątkiem jest perceptron, w przypadku którego funkcja aktywacji nie jest poddana tym ograniczeniom. W praktyce stosuje się najczęściej następujące funkcje: liniowa, logistyczna, tangens hiperboliczny, sinus oraz signum.

- Funkcja liniowa

Neurony z *liniową funkcją aktywacji* tworzą liniowe sieci neuronowe. Wartości ważonej sumy wejść są przepisywane na wyjście neuronu. Budowanie wielowarstwowych sieci liniowych nie jest uzasadnione, ponieważ zawsze można zastąpić ją odpowiednią siecią jednowarstwową.

- Funkcja logistyczna

Funkcja *logistyczna (sigmoidea)* jest bardzo często stosowaną funkcją aktywacji neuronu. Określona jest ona następującym wzorem:

$$f(s) = \frac{1}{1 + e^{-\beta s}} \quad (3.6)$$

Jest to funkcja *unipolarna* (przyjmująca wartości z przedziału $(0,1)$).

- Funkcja tangens hiperboliczny

Równie często stosowaną funkcją przejścia jest *tangens hiperboliczny*. Wyraża się on wzorem:

$$f(s) = \tanh(s) = \frac{1 - e^{-\beta s}}{1 + e^{-\beta s}} \quad (3.7)$$

Funkcja tangens hiperboliczny przyjmuje wartości z przedziału $(-1,1)$. Jest to funkcja *bipolarna*.

- Funkcja sinus

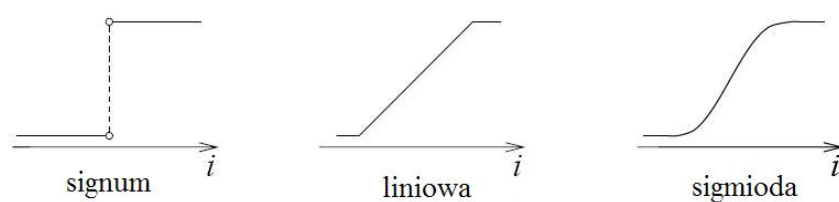
Dla zastosowań w sieciach neuronowych, stosuje się również funkcję *sinus* w następującej postaci:

$$f(s) = \begin{cases} -1 & \text{gdy } s < \frac{-\pi}{2} \\ \sin(s) & \text{gdy } \frac{-\pi}{2} < s < \frac{\pi}{2} \\ 1 & \text{gdy } s > \frac{\pi}{2} \end{cases} \quad (3.8)$$

- Funkcja signum

Funkcja *signum* jest także wykorzystywana.

$$f(s) = \begin{cases} 1 & \text{gdy } s > 0 \\ 0 & \text{gdy } s = 0 \\ -1 & \text{gdy } s < 0 \end{cases} \quad (3.9)$$



Rysunek 3.4: Funkcje aktywacji

Zestawienie nieliniowych funkcji aktywacji.

<i>Funkcja</i>	<i>Wzór funkcji</i>	<i>Wzór pochodnej</i>
Sigmoida	$f(x) = \frac{1}{1+e^{-\beta x}}$	$f'(x) = \beta(1 - f(x))f(x)$
Tangens hiperboliczny	$f(x) = \frac{1-e^{-\beta x}}{1+e^{-\beta x}}$	$f'(x) = \beta(1 - f^2(x))$
Sinusoida	$f(x) = \sin(\beta x)$	$f'(x) = \beta\sqrt{1 - f^2(x)}$
Cosinusoida	$f(x) = \cos(\beta x)$	$f'(x) = -\beta\sqrt{1 - f^2(x)}$
$\frac{x}{1+ x }$	$f(x) = \frac{\beta x}{1+ \beta x }$	$f'(x) = \frac{\beta}{1+ \beta x } - \frac{\beta x}{(1+ \beta x)^2}$

Rysunek 3.5: Nieliniowe funkcje aktywacji i ich pochodne. (Na podstawie [11] Tabela 1)

Współczynnik β wpływa bezpośrednio na stromość funkcji aktywacji, co skutkuje szybkością uczenia całej sieci. Przyjmuje on wartości w przedziale 0.01 – 5.0, najczęściej od 0.8 do 2.0 [11].

3.4. Modele sztucznych neuronów

W sztucznych sieciach neuronowych stosowane są różne modele neuronów. Przeważnie różnią się one funkcją aktywacji oraz algorytmem uczenia.

3.4.1. Perceptron

Jak już zostało wspomniane wyżej, pierwszym modelem sztucznego neuronu był *perceptron*. Jego działanie można zapisać za pomocą wzoru:

$$y = f\left(\sum_{j=1}^N x_j w_j + \theta\right) \quad (3.10)$$

Łatwo zauważyć iż wzór (3.10) koreponduje z zapisem (3.3) gdy $\theta = w_0$. Funkcja f może być nieciągłą funkcją skokową bipolarną (przyjmującą wartości 1 i -1) lub unipolarną (przyjmującą wartości 0 oraz 1). Ze względu na postać funkcji aktywacji, perceptron przyjmuje tylko dwie wartości wyjściowe. Z tego powodu może on klasyfikować sygnały podane na wejściu do jednej z dwóch klas. Perceptron z jednym wejściem może ocenić, czy sygnał wejściowy jest dodatki czy ujemny. Perceptron z dwoma wejściami dzieli płaszczyznę na dwie części. Podział ten wyznacza prosta

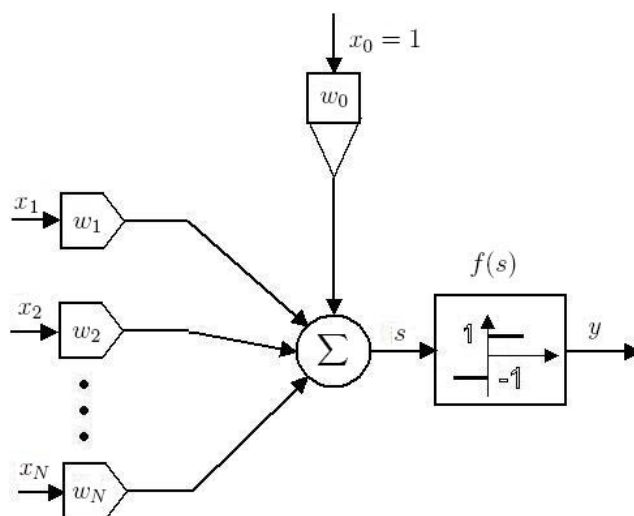
$$w_1 x_1 + w_2 x_2 + \theta = 0. \quad (3.11)$$

Jeżeli perceptron ma n wejść, dzieli n -wymiarową przestrzeń wektorów wejściowych x na półprzestrzenie. Przestrzenie te są rozdzielone $n - 1$ -wymiarową hiperpłaszczyzną (*granicą decyzyjną*) daną wzorem [1]6.2.2., s.169 :

$$\sum_{j=1}^N w_j x_j + \theta = 0. \quad (3.12)$$

Podczas nauki perceptronu jego wagi są modyfikowane. Proces ten odbywa się za pomocą algorytmów zwanych *uczeniem z nauczycielem* lub *uczeniem nadzorowanym*. Istnieją również

algorytmy uczenia *bez nauczyciela* lub *uczenia nienadzorowanego* [1] 6.5, s.213. Polegają one na odpowiednim wzmacnianiu lub osłabianiu reakcji neuronu na sygnał wejściowy.



Rysunek 3.6: Model perceptronu. (Na podstawie [1] 6.2.2., s.169)

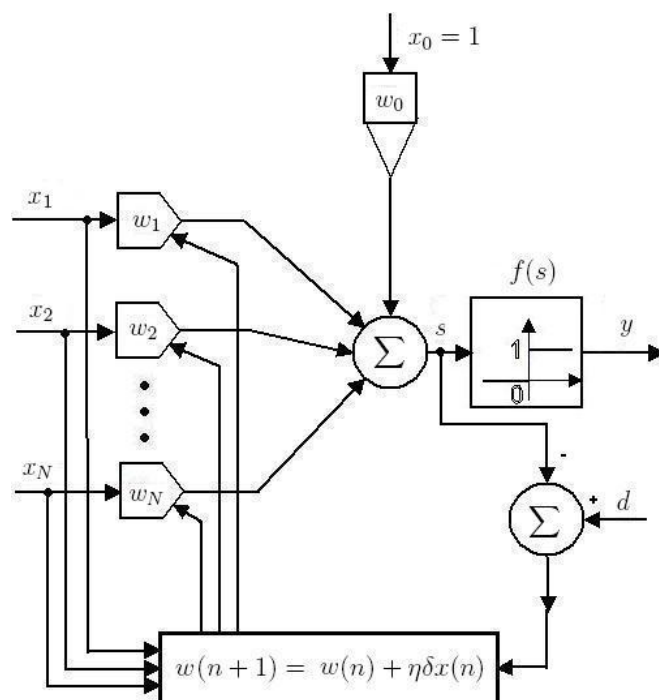
3.4.2. Model Adeline

Model neuronu Adeline (ang. *Adaptive Linear Neuron*) jest bardzo podobny do modelu perceptronu. Różnią się one sposobem uczenia. Neurony Adeline uczone są za pomocą reguły *delta*. Sposób wyznaczanie sygnału wyjściowego jest identyczny z perceptronem. Jednak Adeline porównuje sygnał wzrocowy d z sygnałem s na wyjściu części liniowej neuronu (*sumator*, nie uwzględnia się funkcji aktywacji neuronu).

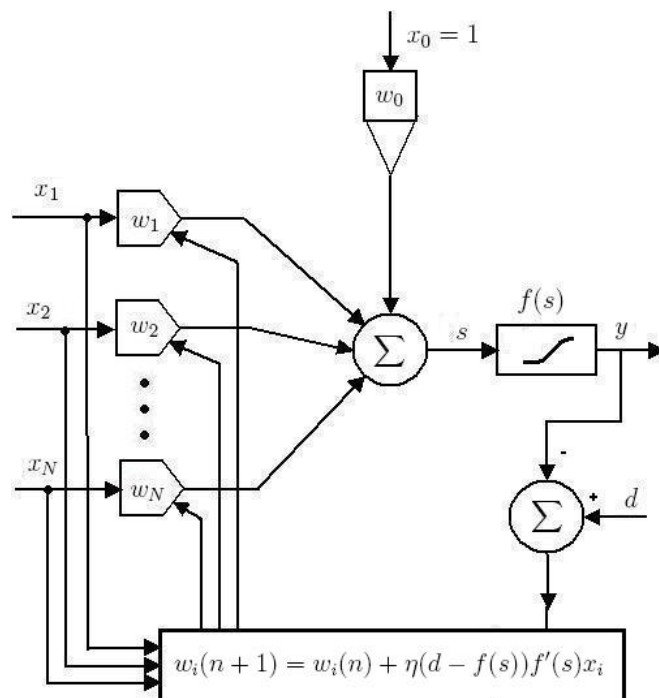
Neurony typu Adeline mogą być również nauczane za pomocą *rekurencyjnej metody najmniejszych kwadratów* (ang. *Recursive Least Squares - RLS*).

3.4.3. Model neuronu sigmoidalnego

Budowa neuronu sigmoidalnego jest analogiczna jak budowa perceptronu oraz modelu Adeline. Różnica polega na zastosowaniu innej funkcji aktywacji. Funkcja ta przybiera postać funkcji sigmoidalnej (unipolarnej lub bipolarnej). Dużą zaletą tej funkcji jest możliwość wprowadzenia parametru β . Przy małej wartości współczynnika β funkcja ma kształt łagodny, wraz ze wzrostem współczynnika wykres staje się coraz bardziej stromy, aż funkcja przyjmuje charakter progowy. Ogromną zaletą funkcji sigmoidalnych jest ich różniczkowalność. W modelu neuronu sigmoidalnego, podobnie jak w modelu Adeline do uczenia użyta jest reguła *największego spadku*. Jednak w przeciwieństwie do Adeline, uwzględniona zostaje funkcja aktywacji. Neurony sigmoidalne można również uczyć metodą najmniejszych kwadratów (RLS).



Rysunek 3.7: Model adeline. (Na podstawie [1] 6.2.3., s.175)



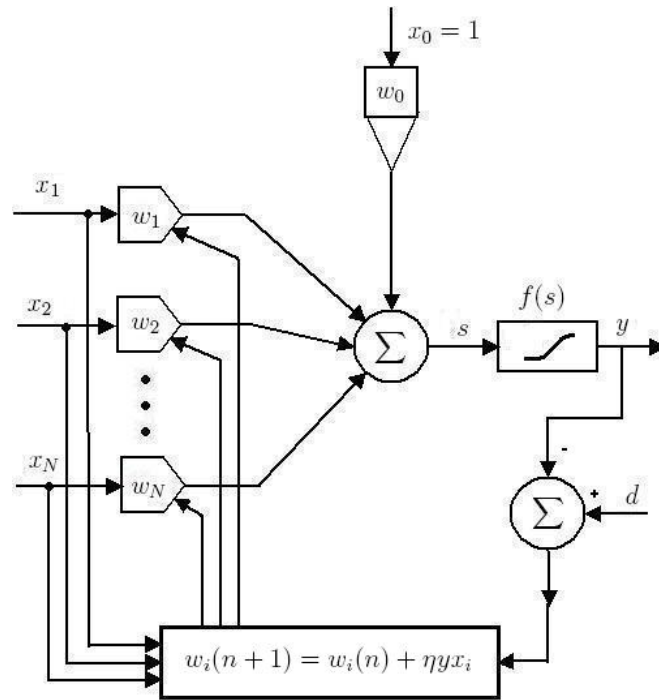
Rysunek 3.8: Model neuronu sigmoidalnego. (Na podstawie [1] 6.2.4., s.180)

3.4.4. Model neuronu Hebba

Model neuronu *Hebba* ma identyczną strukturę jak model Adeline czy sigmoidalny. Tak samo jak w innych przypadkach, rozróżnia go sposób nauki. Metody uczenia stosowane w tych

neuronach nazwane są *regułami Hebba*. Reguła Hebba może przyjmować postać bez nauczyciela lub z nauczycielem. Hebb badał komórki nerwowe. Zauważył on, iż połączenia pomiędzy dwoma komórkami są wzmacniane wtedy, kiedy w tym samym czasie obie komórki stają się aktywne (oba neurony zapalają się). Zgodnie ze swym odkryciem zaproponował on algorytm nauki sztucznych neuronów. W przypadku pojedynczego neuronu wartość wag w_j modyfikuje się proporcjonalnie do wartości sygnału podanego na j -te wejście jak i sygnału wyjściowego z uwzględnieniem współczynnika uczenia η .

Wadą algorytmu nauczania według reguł Hebba jest to, iż wartości wag mogą wzrastać do dowolnie dużych liczb. Z tego powodu wprowadzone zostały modyfikacje reguły Hebba [1] 6.2.5., s.184-185.

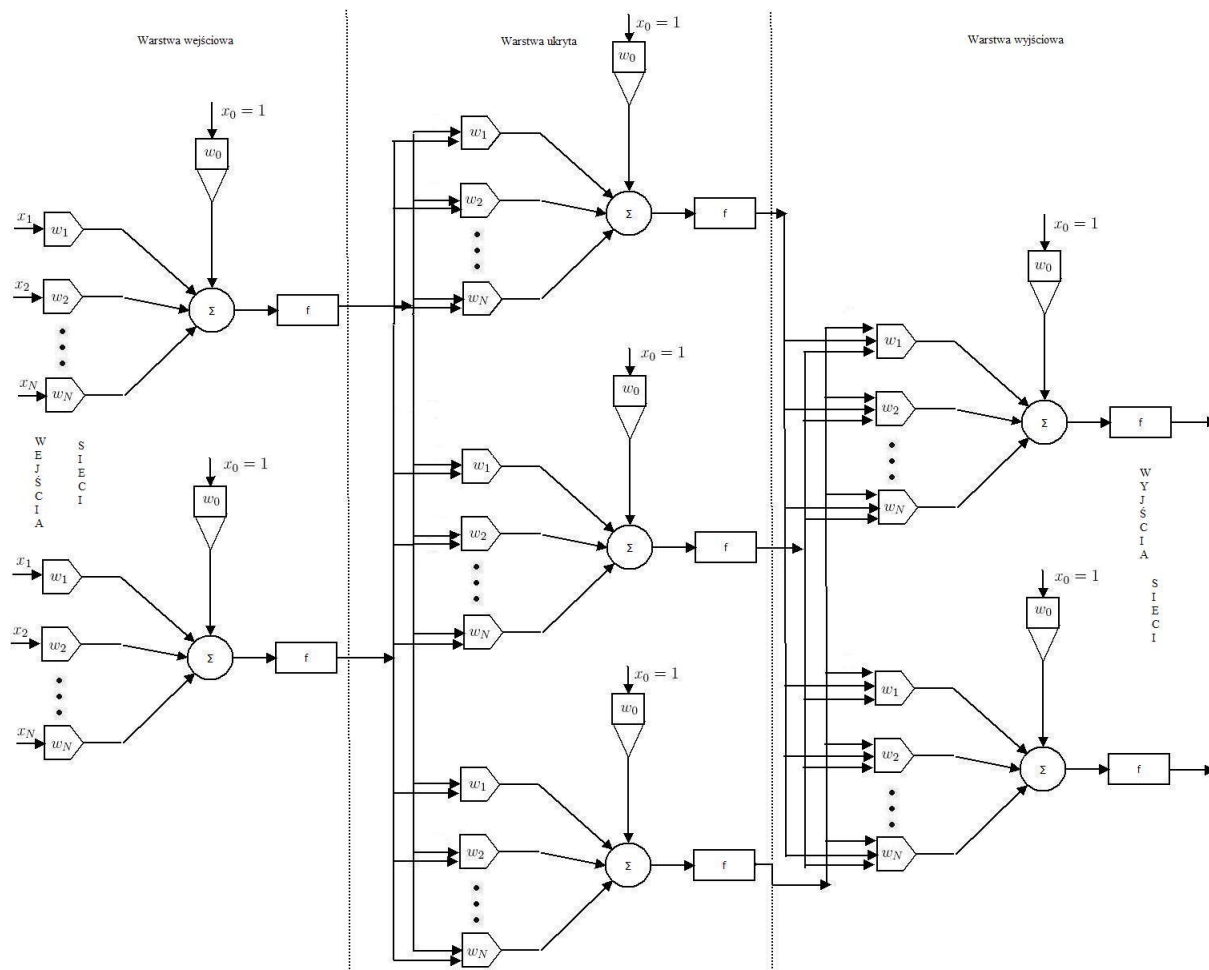


Rysunek 3.9: Model neuronu Hebba. (Na podstawie [1] 6.2.5., s.184)

3.5. Struktura i podstawy działania sieci

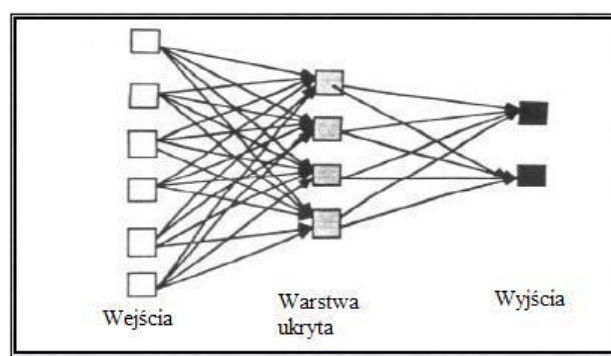
Pojemność informacyjna pojedynczego neuronu nie jest duża. Powiększenie pojemności i zwiększenie zdolności przetwarzania uzyskuje się poprzez odpowiednie połączenie wielu neuronów. W taki sposób powstaje sztuczna sieć neuronowa. Powstające w ten sposób warstwy sieci można podzielić na:

- warstwę wejściową
- warstwy ukryte
- warstwę wyjściową



Rysunek 3.10: Schemat budowy jednokierunkowej, nieliniowej sztucznej sieci neuronowej.

Działanie takiej sieci neuronowej przedstawia się następująco. Na wejścia pierwszej warstwy (warstwy wejściowej) wprowadzane są dane wejściowe. Następnie, dane (już jako dane wyjściowe pierwszej warstwy - warstwy wejściowej) przekazywane są na wejścia kolejnych warstw, aż do warstwy wyjściowej. Wartości otrzymane w ostatniej, wyjściowej warstwie sieci neuronowej traktowane są jako wyjścia sieci.



Rysunek 3.11: Uproszczony schemat budowy wielowarstwowej sztucznej sieci neuronowej. (Na podstawie [7] 4.2., s.5)

3.6. Uczenie sieci

Decydującą zaletą sieci neuronowej w porównaniu do innych metod przetwarzania danych jest umiejętność generalizacji wiedzy, co umożliwia poprawną reakcję na sygnały nie przewidziane przez projektanta.

Dodatkowym atutem w odróżnieniu od metod matematycznych lub algorytmicznych jest możliwość wykorzystywania sieci dla wielu różnych modeli, bez konieczności wprowadzania znaczących modyfikacji.

Podane cechy zostaną uzyskane tylko poprzez zastosowanie odpowiedniego algorytmu uczenia. Istnieje wiele różnych algorytmów uczenia sieci. Najczęściej stosowana jest metoda wstecznej propagacji błędów oraz jej modyfikacje.

3.6.1. Algorytm wstecznej propagacji błędów

Algorytm wstecznej propagacji błędów jest podstawowym algorytmem uczenia nadzorowanego (z nauczycielem) wielowarstwowych jednokierunkowych sieci neuronowych.

W algorytmie wstecznej propagacji błędów wykorzystywane są dwa ciągi danych: ciąg uczący oraz ciąg weryfikujący. Ciąg uczący, inaczej wektor uczący składa się z wektora wejściowego oraz wektora wyjściowego. Po przetworzeniu wektora wejściowego, otrzymany wynik porównywany jest z wartościami oczekiwanymi. Jeżeli odpowiedź nie jest poprawna, liczony jest błąd odpowiedzi. Błąd ten propagowany jest do sieci w odwrotnej kolejności niż wektor wejściowy. Na podstawie błędu modyfikowane są wagi. Dana procedura powtarzana jest aż do momentu otrzymania błędu mniejszego niż założony.

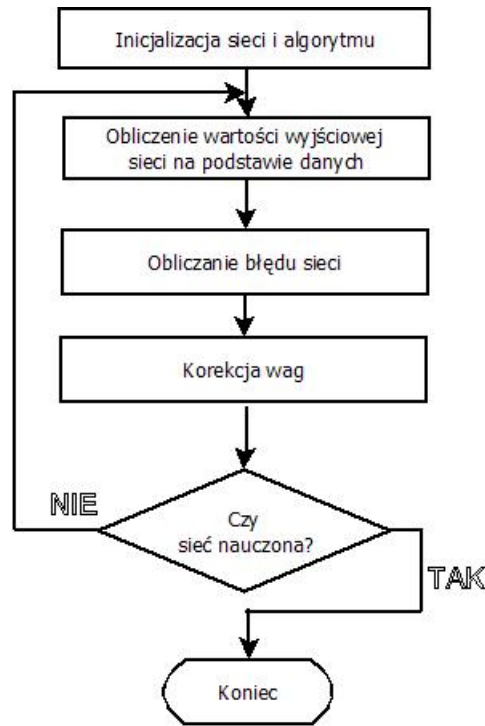
Po przetworzeniu całego ciągu uczącego (*epoce*) obliczany jest błąd epoki. Czynności powtarzane są aż do momentu otrzymania minimalnego błędu.

Przebieg uczenia nadzorowanego sieci neuronowej przedstawiony jest na rysunku 3.12.

Jak już wspomniano wyżej, sieć uczona jest poprzez odpowiednie zmiany wartości wag na wejściach poszczególnych neuronów. W tym celu niezbędne jest dokładne określenie wartości błędów na wyjściach poszczególnych neuronów. Wykorzystywane są następujące wzory.

Dla $k = N$, czyli warstwy ostatniej:

$$\delta_i(n) = \epsilon_i(n)f'(s_i(n)) \quad (3.13)$$



Rysunek 3.12: Schemat przebiegu algorytmu wstecznej propagacji błędu. (Na podstawie [12] 1.6.1.)

gdzie

$\epsilon_i(n)$ jest błędem sygnału wyjściowego danego neuronu

$$\epsilon_i(n) = d_i(n) - y_i(n) \quad (3.14)$$

$s_i(n)$ jest sygnałem wyjściowym części liniowej neuronu

$$s_i(n) = \sum_{j=0}^{N_{k-1}} w_{ij}(n)x_j(n) \quad (3.15)$$

$d_i(n)$ jest sygnałem wzorcowym i -tego neuronu

y_i jest wyjściem danego neuronu

$w_{ij}(k)$ jest wagą wejścia j w i -tym neuronie w warstwie k

n jest numerem okresu przebiegu algorytmu.

Dla każdej poprzedniej warstwy k błąd obliczany jest według następującego wzoru:

$$\delta_i^{(k)}(n) = f'(s_i^{(k)}(n)) \sum_{m=1}^{N_{k+1}} \delta_i^{(k+1)}(n) w_{mi}^{(k+1)}(n) \quad (3.16)$$

Po wyliczeniu błędów, następuje modyfikacja wag według wzoru:

$$w_{ij}^{(k)}(n+1) = w_{ij}^{(k)}(n) + \eta \delta_i^{(k)} x_j^{(k)} \quad (3.17)$$

gdzie η jest współczynnikiem uczenia.

Po podstawieniu otrzymujemy [12] 1.6.1.:

$$w_{ij}^{(k)}(n+1) = w_{ij}^{(k)}(n) + \eta \epsilon_i^{(k)}(n) f'(s_i^{(k)}(n)) x_j^{(k)} \quad (3.18)$$

W przedstawionej metodzie uczenie sieci (modyfikacja wag) może być przeprowadzana każdorazowo po podaniu na wejście wektora uczącego (*przyrostowe uaktualnianie wag*). Możliwe jest również kolejne podawanie wektorów uczących na wejście sieci, a następnie wyznaczanie odpowiadających im sygnałów na wyjściu sieci. W kolejnym kroku po porównaniu z wartościami wzorcowymi otrzymane w kolejnych iteracjach błędy są sumowane. Po zakończeniu podawania na wejście próbek z całej epoki dokonywana jest korekta wartości wszystkich wag z wykorzystaniem skumulowanej wartości błędu (*kumulacyjne uaktualnianie wag*).

Niestety, algorytm wstecznej propagacji ma kilka wad. Do najczęściej wymienianych należy duża liczba iteracji potrzebna do osiągnięcia oczekiwanego wyniku oraz wrażliwość na minima lokalne funkcji błędu. Działanie tego algorytmu jest również uzależnione od odpowiednio dobranego współczynnika szybkości uczenia η . Wpływa on na szybkość uczenia poprzez bezpośrednie działanie na gradient zmiany. Zbyt duży współczynnik η powoduje poruszanie się po wierzchołkach płaszczyzny błędu i pomijanie zagłębień z minimami. Zbyt mała wartość może być przyczyną utknięcia procesu uczenia w minimum lokalnym. Znane są techniki pozwalające na dynamiczny dobór wartości tego współczynnika w zależności od aktualnego postępu uczenia. Najczęściej współczynnik uczenia η przyjmuje wartości od 0.01 do 0.6 [11].

Istnieją jednak sposoby na zwiększenie tempa uczenia bez pogorszenia jakości uczenia niewykazujące aż takiej wrażliwości na minima lokalne.

Jedną z takich metod jest *algorytm wstecznej propagacji błędów ze współczynnikiem momentum*.

3.6.2. Algorytm wstecznej propagacji błędów ze współczynnikiem momentum

Algorytm wstecznej propagacji błędów ze współczynnikiem momentum jest modyfikacją algorytmu wstecznej propagacji błędów polegającą na wprowadzeniu dodatkowego współczynnika α (zwanego *momentum*) zgodnie z którym wzór (3.16) modyfikowany jest następująco [12] 1.6.2.:

$$w_{ij}^{(k)}(n+1) = w_{ij}^{(k)}(n) + \eta \delta_i^{(k)}(n) x_j^{(k)} + \alpha [w_{ij}^{(k)}(n) - w_{ij}^{(k)}(n-1)] \quad (3.19)$$

Współczynnik α pełni rolę współczynnika bezwładności przy modyfikacji wagi. Powoduje to większą płynność zmian oraz "przeskakiwanie" nad minimami lokalnymi.

Współczynnik *momentum* uzależnia wartość wagi w następnym kroku ($n+1$) nie tylko od jej wagi w kroku obecnym (n) ale również w kroku poprzednim ($n-1$). Jeśli w poprzednich krokach kierunek modyfikacji wagi był taki sam to zostaje zwiększona wartość modyfikująca wagę, w przeciwnym wypadku współczynnik *momentum* powoduje zahamowanie gwałtownych zmian wag.

Współczynnik α przyjmuje wartości z zakresu $(0, 1)$. Przeważnie $\alpha = 0.9$ [11].

3.6.3. Algorytm zmiennej metryki

Algorytm ten bazuje na minimalizacji funkcji celu (którą jest błąd wag neuronów całej sieci) w rozwinięciu szeregu Taylora do trzeciego składnika.

Algorytm ten stosuje się do niedużych sieci, gdyż wykazuje on stosunkowo dużą złożoność obliczeniową. Jego zaletą jest dość szybka zbieżność.

3.6.4. Algorytm Levenberga- Marquardta

Algorytm ten również bazuje na minimalizacji funkcji celu w rozwinięciu szeregu Taylora do trzeciego składnika. Ponadto jest tu jeszcze wprowadzony tzw. *parametr Levenberga-Marquardta* μ . Parametr ten dobierany jest w zależności od błędu na wyjściu sieci podczas procesu uczenia. Przyjmuje on duże wartości na stracie algorytmu, jego wartość maleje aż do wartości zerowej w miarę zbliżania się do rozwiązania optymalnego.

3.6.5. Rekurencyjna metoda najmniejszych kwadratów

Rekurencyjna metoda najmniejszych kwadratów jest rozwinięciem metody wstecznej propagacji błędów z członem momentum dla sieci wielowarstwowych.

Wprowadzony jest tutaj *współczynnik zapominania* λ wybierany z przedziału $(0, 1]$.

3.7. Rodzaje sieci

Powyżej przedstawiony został najprostszy model sieci neuronowej (jednokierunkowa sieć wielowarstwowa). Istnieje jednak wiele innych rodzajów sieci neuronowych. Ich podziału można dokonać według kilku kryteriów.

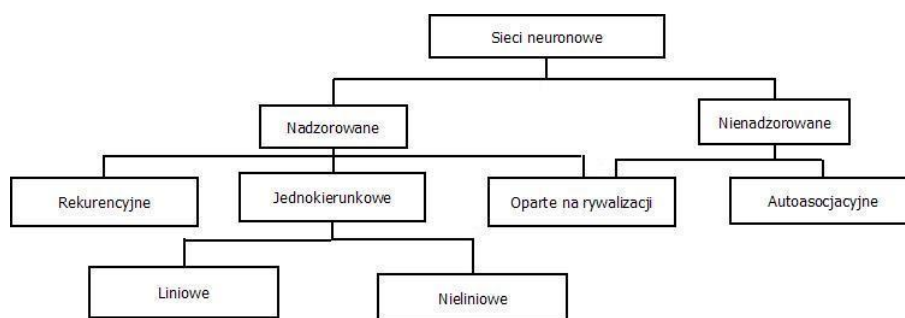
Poniżej przedstawiam cztery podstawowe kryteria klasyfikacji sieci neuronowych:

- metoda trenowania
- kierunek propagacji sygnałów w sieci
- typ funkcji przejścia (funkcji aktywacji)
- rodzaj danych wprowadzanych na wejścia

Klasyfikacja sieci według tych kryteriów wygląda następująco:

Często spotykanymi sieciami są sieci *nadzorowane*. Ich trening polega na prezentowaniu sieci wartości wejściowych wraz z odpowiednimi, pożądanymi wartościami wyjściowymi. Na tej podstawie, sieć ustawia wartości wag aby odpowiedź mieściła się w granicy błędu, który określa użytkownik.

Sieci *liniowe* (z liniową funkcją aktywacji - Rozdział 3.3.1) stosowane są rzadziej, z powodu małego zakresu możliwości.



Rysunek 3.13: Klasyfikacja sieci neuronowych. (Na podstawie [12] 1.4.)

Najczęściej stosowanymi obecnie strukturami nerwowymi są sieci *jednokierunkowe nieliniowe*. Ich zalety to dobra efektywność działania oraz prosta i intuicyjna budowa. Są one także bardzo uniwersalne.

Sieci *rekurencyjne* charakteryzują się innym sposobem działania (a zatem i inną budową) niż sieci *jednokierunkowe*. Wyjścia niektórych neuronów, lub całych warstw mogą być wprowadzane ponownie na wejścia tych samych neuronów lub warstw.

Sieci *nie nadzorowane* różnią się od sieci *nadzorowanych* sposobem treningu. Nie wymagają one podawania pożądaných wartości wyjściowych dla prezentowanych wejść lecz tylko wektorów wejściowych.

Wybrane typy sieci neuronowych

Poniżej przedstawiam zestawienie oraz przykłady wybranych typów sieci neuronowych.

- Warstwowe sieci liniowe
 - Adeline / Madeline
- Wastwowe sieci nieliniowe
 - sieć wielowarstwowa uczona algorytmem wstecznej propagacji błędów
 - sieci wielowarstwowe z modyfikacjami algorytmu wstecznej propagacji błędów
 - sieci z funkcjami o symetrii kołowej - RBF
- Sieci ze sprzężeniem zwrotnym (sieci rekurencyjne)
 - sieci Hopfielda
 - dwukierunkowa pamięć asocjacyjna - BAM
- Sieci uczone przez współzawodnictwo
 - sieci Kohonena, LVQ
 - odwzorowanie cech istotnych oraz sieci samoorganizujące się SOM
- Sieci rezonansowe oraz sieci z kontrpropagacją
- Sieci neuronowe zintegrowane z algorytmami metaheurystycznymi
 - symulowanie wyżarzanie - maszyna Boltzmana
 - algorytmy genetyczne
- Metody hybrydowe wykorzystujące sieci neuronowe

- Systemy rozmyto- neuronowe

3.8. Dobór architektury sieci

Jedną z ważniejszych cech sieci neuronowej, wpływających na jej parametry działania takie jak szybkość uczenia, wielkość popełnianego błędu oraz zdolność generalizacji, ma odpowiednio dobrana struktura sieci. Niestety nie istnieją jednoznaczne metody jej definiowania. Przeważnie ilość neuronów w warstwach wejściowej oraz wyjściowej jest determinowana przez założenia użytkownika (a więc przez dany problem). Niestety ilość warstw ukrytych, a także neuronów, z ilu warstw te się składają nie są wielkościami oczywistymi.

Dobór struktury sieci neuronowej polega więc na doborze liczby warstw sieci oraz doborze liczby neuronów w każdej warstwie. Wielkości te uzależnione są od problemu jaki dana sieć ma za zadanie rozwiązać.

Jak już wspomniano powyżej, pojedynczy neuron dzieli płaszczyznę na dwie części. Dwie warstwy neuronów są już w stanie odwzorować simpleksy (obszary wypukłe ograniczone hiperpłaszczyznami). Mając do dyspozycji trzy warstwy, można zdefiniować dowolny obszar. Wynika z tego, iż trójwarstwowa sieć powinna rozwiązać szeroką gamę problemów klasyfikacji oraz aproksymacji.

Zostało stwierdzone, iż sieć neuronowa z jedną warstwą ukrytą o neuronach z sigmoidalną funkcją aktywacji oraz z liniowym neuronem wyjściowym jest w stanie aproksymować dowolną rzeczywistą funkcję ciągłą zdefiniowaną na $[0, 1]^n$. W praktyce okazuje się, iż istnieje niewiele problemów, do których rozwiązania potrzeba więcej niż dwóch warstw ukrytych.

Znaczący wpływ na skuteczność sieci ma również liczba neuronów w poszczególnych warstwach. Zbyt duża liczba neuronów może wydłużyć proces uczenia sieci. Jeśli liczba próbek uczących jest niewielka w porównaniu do rozmiaru sieci, istnieje możliwość „przeuczenia” sieci. Sieć traci wtedy zdolność uogólniania wiedzy i nie potrafi reagować na wzorce nie zawarte w próbce uczącej. Zbyt mała liczba neuronów w stosunku do liczby próbek uczących powoduje, że sieć źle przewiduje (np. źle aproksymuje daną funkcję).

Aby problem ten rozwiązać, powstało szereg metod optymalizacji wielowarstwowych sieci neuronowych. Algorytmy te można podzielić na trzy grupy:

- metody wzrostu [12] 1.7.2.,
- metody redukcji [12] 1.7.3.,
- metody optymalizacji dyskretniej [12] 1.7.4..

Rozdział 4

Opis projektu

4.1. Inspiracje

Inspiracją projektu były coraz to bardziej rozpowszechnione komputery-zabawki dla dzieci. Według moich przypuszczeń, w przyszłości pismo ręczne zaniknie, gdyż już teraz w wielu szkołach podstawowych od najmłodszych lat wymagany jest komputer osobisty. Większość prac pisemnych przygotowywana jest przy użyciu komputera. Pismo ręczne przeważnie używane jest tylko do podpisów (co również może zaniknąć, podpis ręczny zostanie wyparty przez podpis cyfrowy).

Można jednak wykorzystać wszechobecność komputerów dla rozwoju dziecka, nie tylko jego zdolności umysłowych lecz także motorycznych (czym jest proces pisania).

4.2. Założenia projektu

Tworzony przeze mnie program ma charakter edukacyjny. Jest to program tworzony z myślą o dzieciach, pomocny w nauce przede wszystkim pisania. Dzięki niemu dziecko nie potrzebuje nadzoru rodziców ani nauczyciela. Program powinien rozpoznać symbol danego alfabetu (literę lub cyfrę), jeśli jest dobrze napisany, tzn. w wystarczającym stopniu przypomina jeden z symboli alfabetu. Program potrafi również sprawdzić poprawność wyniku działania matematycznego.

4.3. Analiza biznesowa

4.3.1. Słownik

Użytkownik - osoba używająca system, przeważnie będzie to dziecko.

Symbol - litera alfabetu lub cyfra arabska.

Działanie matematyczne - niespomplikowane działanie matematyczne (takie jak dodawanie, odejmowanie).

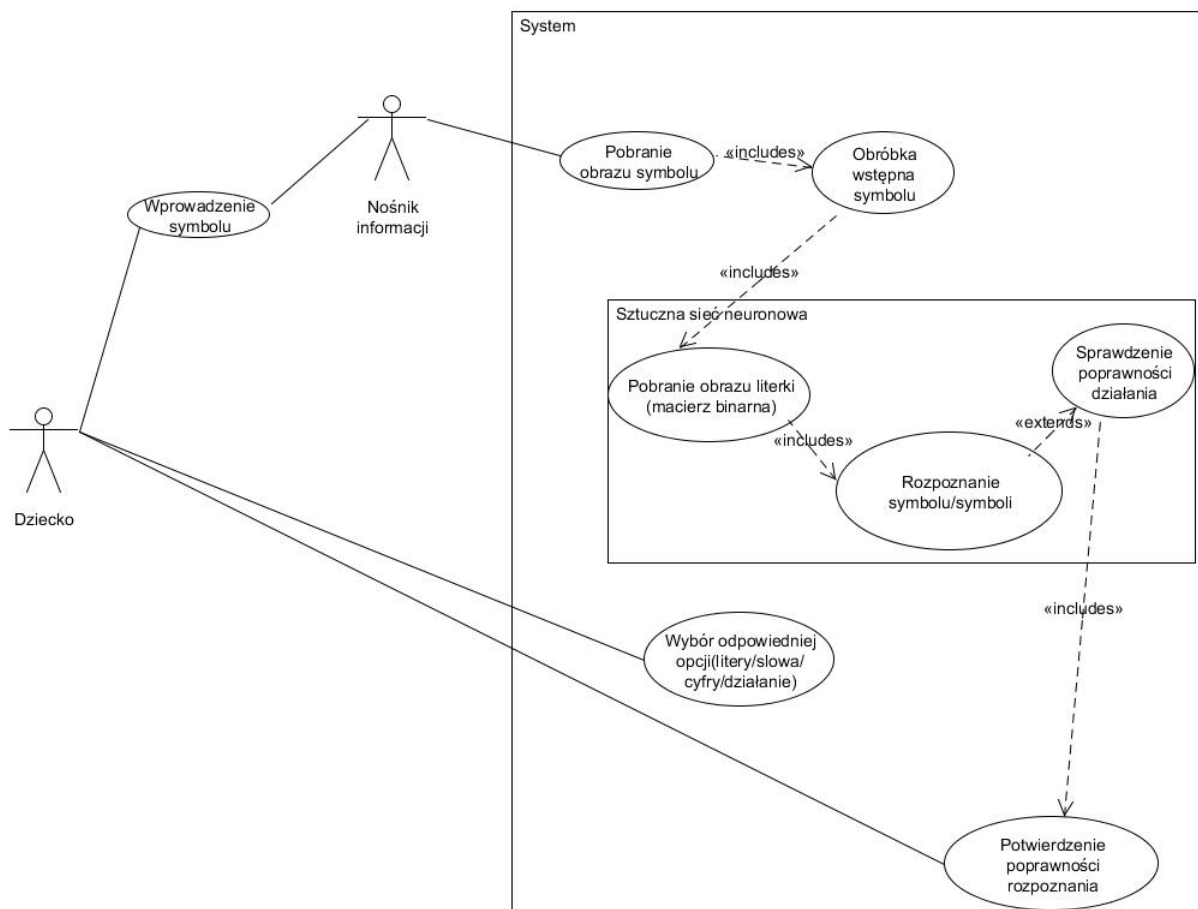
Sieć neuronowa - użyta w systemie sieć neuronowa.

Neuron - sztuczny neuron, jednostka sieci neuronowej.

Dane wejściowe - dane podawane na wejściu sieci neuronowej, macierz binarna.

Dane wyjściowe - konkretne symbole, bądź wynik działania.

4.3.2. Use case



Rysunek 4.1: Przypadki użycia

4.3.3. Specyfikacja uzupełniająca

Projekt ma za zadanie:

- dać dziecku możliwość wyboru nauki pisania liter lub cyfr lub liczenia,
- pobrać i przekształcić obraz (dyskretyzacja, binaryzacja, normalizacja),
- rozpoznać obraz (za pomocą sztucznej sieci neuronowej),
- zwrócić rozpoznany symbol (literę lub cyfrę),
- rozpoznać całe działanie matematyczne oraz sprawdzić poprawność wyniku działania.

Projekt składa się z następujących modułów:

- moduł rozpoznawania obrazu symbolu (sprowadzania obrazu do macierzy binarnej),
 - moduł pobierania obrazu symbolu,
 - moduł przetwarzania obrazu sybolu,
- moduł rozpoznawiania pobranego obrazu symbolu - sztuczna sieć neuronowa,
- moduł komunikacji z użytkownikiem (graficzny interfejs użytkownika GUI).

Rozpoznawanie obrazu symbolu:

- dane wejściowe w postaci pliku *.jpg*, *.png*, *.bmp*,
- przetwarzanie danych za pomocą funkcji w Matlabie,
- moduł ten ma za zadanie przetworzenie obrazu na macierz binarną (a następnie na wektor).

Sztuczna sieć neuronowa:

- uczona jest metodą wstecznej propagacji błędów ze współczynnikiem momentum,
- posiada jedną warstwę ukrytą,
- w warstwie wejściowej posiada tyle neuronów, na ile komórek dzielimy obraz wejściowy.

Moduł komunikacji z użytkownikiem:

- musi być dość intuicyjny (użytkownikiem jest dziecko),
- pobiera dane od użytkownika,
- zwraca odpowiedź.

4.3.4. Zatwierdzone wymagania

- System powinien dać użytkownikowi wybór pomiędzy poszczególnymi funkcjonalnościami (rozpoznawanie konkretnych symboli danego alfabetu), czyli którą sieć neuronową zastosujemy.
- System pobiera obraz wejściowy oraz zwraca przetworzony obraz mający odzwierciedlać obraz wejściowy jako znak kodu ASCII.
- System powinien rozpoznawać pojedyncze symbole pisma ręcznego z danego alfabetu (np. małe litery, duże litery, cyfry, znaki).
- System powinien sprawdzać poprawność wykonanego działania matematycznego.
- System umożliwia interakcje z użytkownikiem.
- Administrator systemu powinien mieć możliwość trenowania sieci neuronowych oraz zmiany pewnych parametrów dotyczących sieci (zmiana alfabetu, a co za tym idzie zmiana ilości neuronów w warstwie wyjściowej oraz wejściowej i ukrytej, a także zmiana rozmiaru/ rozdzielczości wprowadzanego obrazu).

Możliwe problemy:

- Obraz nie zostaje rozpoznany przez sieć - źle dobrane parametry sieci neuronowej:
 - zbyt mała rozdzielczość wprowadzanego obrazu (zbyt mało neuronów wejściowych w sieci neuronowej oraz neuronów w warstwie ukrytej),
 - za krótki czas uczenia się sieci - za mało epok,
 - źle dobrane parametry (momentum),
 - zły podział na zbiór uczący i testowy (za mało przykładów).
- Obraz jest nieczytelny:
 - obraz jest niespójny,
 - zbyt dużo szumów,
 - zbyt cienka lub za gruba linia pisma.

4.4. Wybór modelu wytwórczego

Modelem wytwórczym tworzonego przeze mnie projektu jest **tworzenie z gotowych komponentów**.

Wykorzystany przeze mnie model zakłada redukcję nakładów poprzez wykorzystanie podobieństwa tworzonego oprogramowania do wcześniej tworzonych systemów oraz wykorzystanie gotowych komponentów dostępnych na rynku. Jest to bardzo dobre rozwiązanie dla osób piszących pracę indywidualnie.

Uzasadnienie:

- Sieci neuronowe są dość powszechnie stosowane, ich implementację można już znaleźć niemalże w każdym języku programowania, w językach skryptowych itd.
- Nie ma sensu pisanie wszystkiego od nowa podczas, gdy ktoś już to wcześniej zrobił.
- Piszę pracę indywidualnie, przez co nie jestem w stanie zrobić wszystkiego od samego początku.

Etapy procesu:

- **Analiza komponentów** - komponenty do pobierania oraz przetwarzania obrazu (*Image Processing Toolbox*) oraz komponenty powiązane z siecią neuronową (*Neural Network Toolbox*)
- **Modyfikacja wymagań** - Należy odpowiednio zmodyfikować sieć neuronową oraz sposób pobierania obrazu, aby były one ze sobą kompatybilne (przetworzony obraz = wejście sieci neuronowej).
- **Projektowanie systemu z użyciem wielokrotnym** - system wykorzystuje sieć neuronową 4 razy - 4 rodzaje sieci oraz wielokrotnie wykorzystuje komponent pozyskiwania oraz rozpoznawania obrazu.

- **Tworzenie i integracja** - należy połączyć system - wczytywania oraz obróbki obrazu z siecią neuronową, zarówno wejścia sieci jak i wyjścia. Na koniec, cały system należy podłączyć do graficznego interfejsu użytkownika (GUI), aby zapewnić łatwość w obsłudze.

4.5. Instrukcja użytkownika

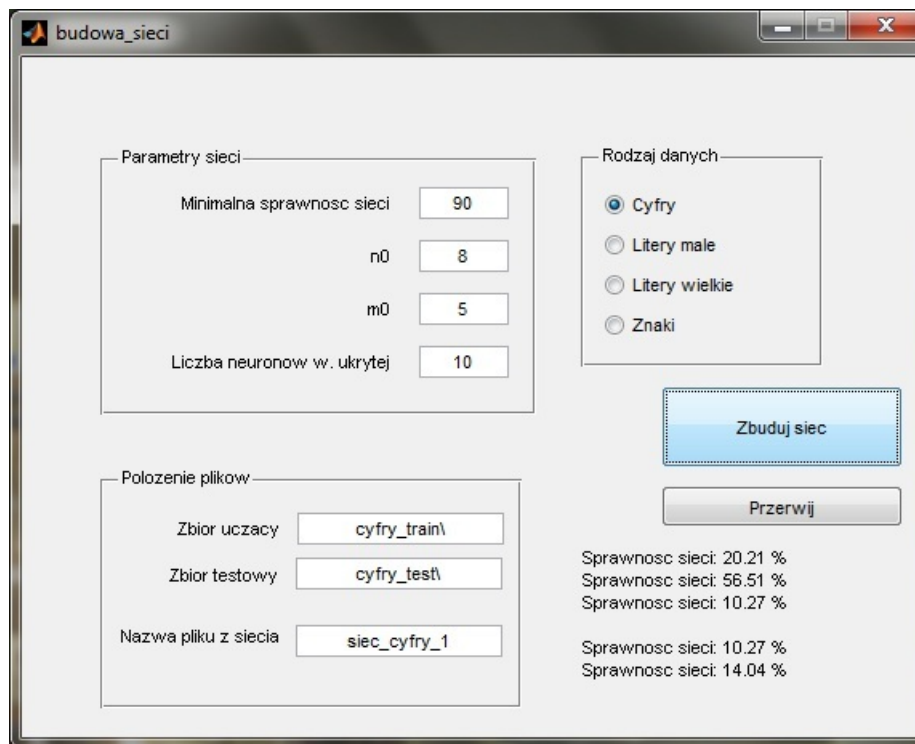
Aby skorzystać z systemu do rozpoznawania pisma należy dysponować programem **MATLAB** w wersji co najmniej MATLAB R2009b zawierający *Image Processing Toolbox* oraz *Neural Network Toolbox*.

4.5.1. Instrukcja dla rodzica

Budowa sieci

- Należy ustawić ścieżkę roboczą na folder w którym znajduje się program.
- *budowa_sieci.m*, *budowa_sieci.fig* – główna aplikacja.
- Tworzymy dwa foldery z danymi uczącymi i testowymi (*Zbior uczacy* oraz *Zbior testowy*), w których umieszczamy pliki graficzne o nazwach zaczynających się zgodnie z prefiksami umieszczonymi w pliku *budowa_sieci.m* (linie od 94 do 114 – zmienne prefix, mogą być modyfikowane).
Na przykład dla wielkich liter tworzymy dwa foldery *wielkie_train* oraz *wielkie_test*, w których następnie zapisujemy obrazy symboli zgodnie z ich nazwami. Przykładowo *A_1.jpg*, *A_2.jpg*, *A_3.jpg*,..., *B_1.jpg*, *B_2.jpg*, *B_3.jpg*,..., *Z_1.jpg*, *Z_2.jpg*, *Z_3.jpg*,.... Podobnie w zbiorze testowym.
- Wywołujemy plik *budowa_sieci*.
- Wybieramy typ danych (cyfry, litery lub znaki).
- Wybieramy parametry nauki, czyli jaka sprawność sieci nam wystarczy (*Minimalna sprawnosć sieci*, od 0 do 100), rozmiar litery po preprocessingu (obróbce wstępnej), czyli na ile pól dzielimy obraz wejściowy **n0** np. 8 (wysokość), **m0** np. 5 (szerokość) oraz liczbę neuronów w warstwie ukrytej (*liczba neuronow w. ukrytej*).
- Wpisujemy nazwy folderów, w których mamy dane uczące i testowe.
Ważne, aby na końcu nazwy dopisać znak "\".
Dla podanego wyżej przykładu będą to *wielkie_train* oraz *wielkie_test*
- Podajemy nazwę pliku, pod jaką zostanie zapisana sieć (*Nazwa pliku z siecia*) *nazwa.mat* i jej statystyka *nazwa.txt*.
- W programie docelowym wykorzystane są *.mat pliki o nazwach:
 - *cyfry*,
 - *litery_m*,
 - *litery_w*,
 - *znaki* (nazwy są zdefiniowane w pliku *dzialania.m*, linie 293, 299, 304, 310).

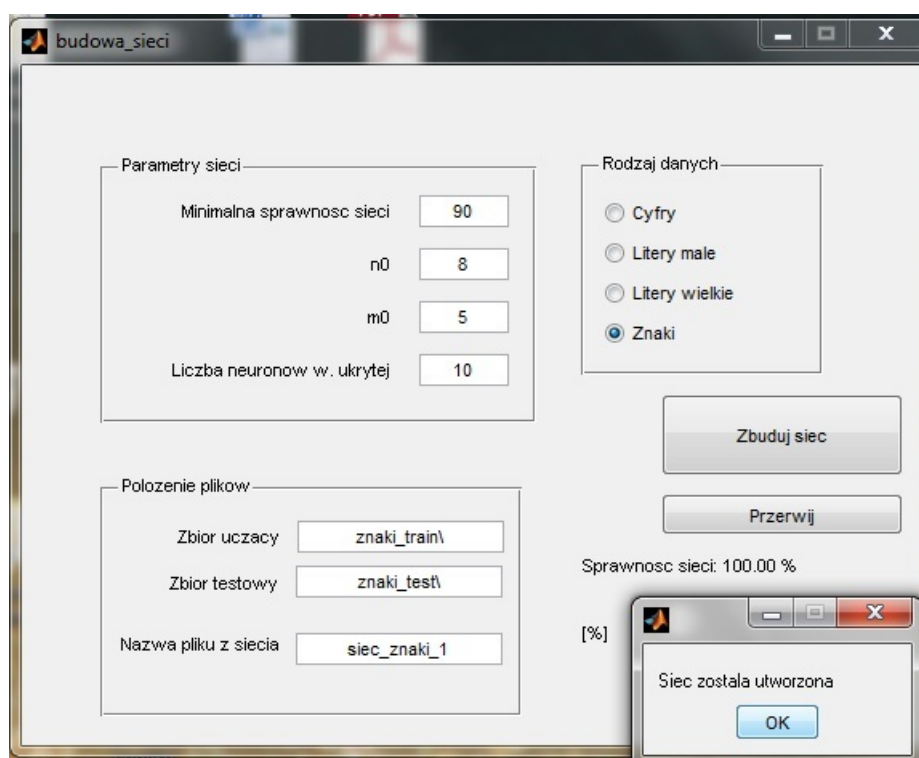
- Klikamy na przycisk *Zbuduj siec*.



Rysunek 4.2: Budowa sieci *cyfry*

- Czekamy na okno z informacją, że sieć została zbudowana (aktualne wyniki testów widać z prawej strony okna).
- Możemy również zakończyć trenowanie sieci klikając przycisk *Przerwij* w dowolnym momencie.

Aby przejść do aplikacji do nauki, należy uruchomić *dzialania.m*.

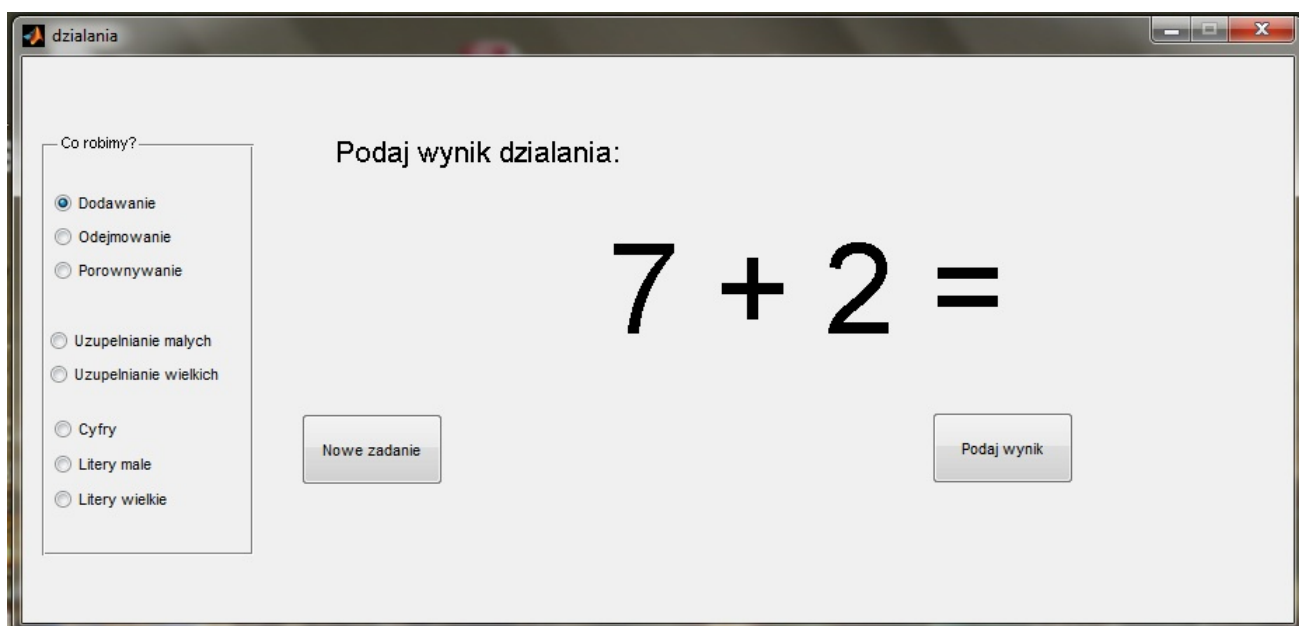


Rysunek 4.3: Pomyślnie utworzona sieć

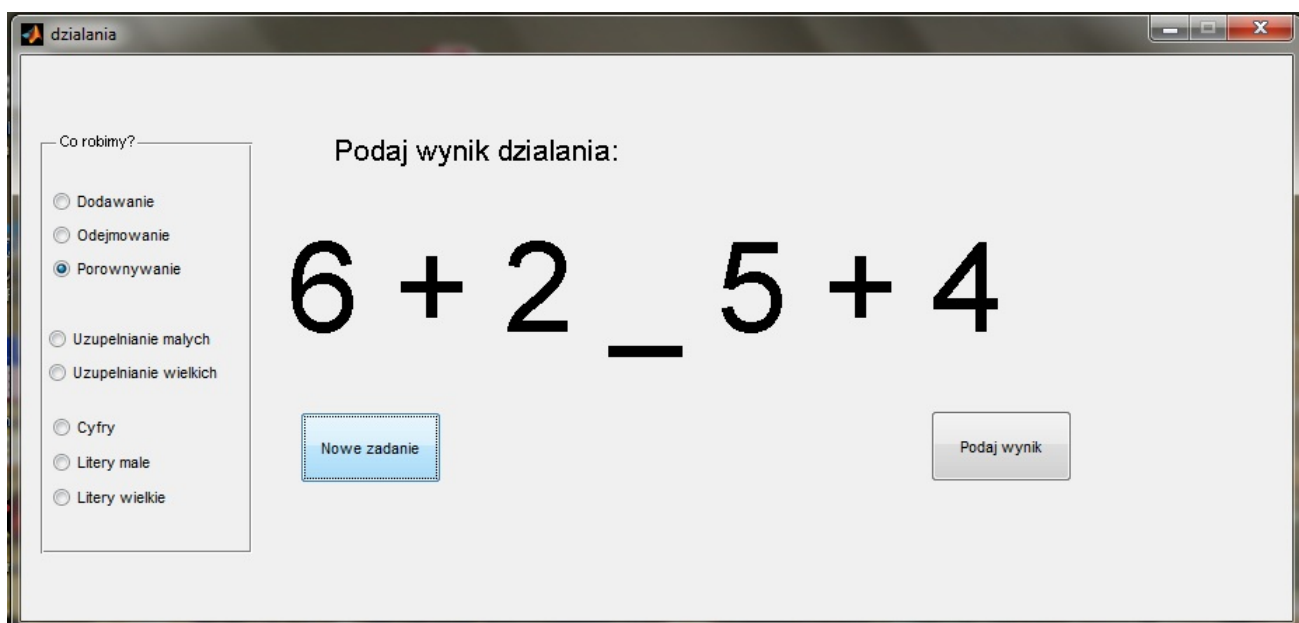
4.5.2. Instrukcja dla dziecka

- Uruchamiany skrypt *dzialania*.
- Pojawia się graficzny interfejs użytkownika.
- Najpierw należy wybrać *Co robimy?*
 - *Dodawanie* - ćwiczenia z dodawania, wpisywanie wyniku.
 - *Odejmowanie* - ćwiczenia z odejmowania, wpisywanie wyniku.
 - *Porównywanie* - ćwiczenia porównywania wyników działań, wpisywanie znaku.
 - *Uzupełnianie małych* - wpisywanie brakujących małych liter do podanego słowa.
 - *Uzupełnianie wielkich* - wpisywanie brakujących wielkich liter do podanego słowa.
 - *Cyfry* - rozpoznawanie cyfr (czy zostały dobrze napisane).
 - *Litere male* - rozpoznawanie małych liter (czy zostały dobrze napisane).
 - *Litere wielkie* - rozpoznawanie wielkich liter (czy zostały dobrze napisane).
- Wybieramy interesującą nas opcję.
- Pojawia się zadanie lub symbol do powielenia.
- Należy wcisnąć przycisk *Podaj wynik*.
- Wyświetli się pytanie o odpowiedź *Jaką liczbę, cyfrę chciałeś zapisać?*

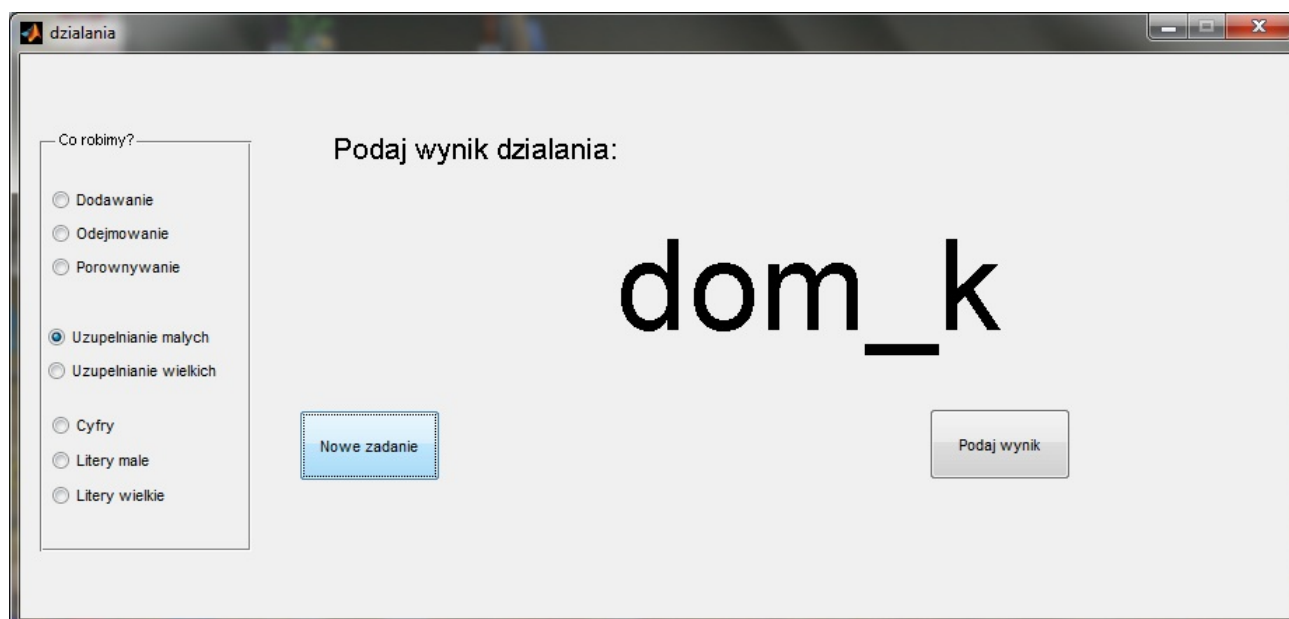
- W odpowiedzi trzeba wprowadzić z klawiatury liczbę, cyfrę lub znak (brakujący symbol lub ten, który chcemy napisać) i wcisnąć *OK* (lub *Cancel* aby skasować wprowadzony symbol).
- Jeśli zaznaczyliśmy cyfry, a wpisujemy małą lub dużą literę to wyskoczy błąd (tak samo w przypadku małych i wielkich liter, znaków oraz na odwrót).
- Następnie należy wprowadzić obraz danego symbolu (cyfry, litery lub znaku o rozszerzeniu *.png* lub *.bmp* lub *.jpg*).
- System sprawdzi poprawność zarówno symbolu wprowadzonego z klawiatury jak i wczytanego obrazu oraz ich zgodność (czyli np. jeśli mamy $1+2=$ i podamy 3 oraz wprowadzimy liczbę 3, która zostanie rozpoznana jako 3, to udzieliliśmy 2 poprawnych odpowiedzi).
- Mamy zatem następujące możliwości:
 - *Chodziło właśnie o ten symbol(cyfre, litere). Symbol (cyfra, litera) był dokładnie napisany* dobrze wprowadzona odpowiedź oraz obraz symbolu.
 - *Chodziło właśnie o ten symbol(cyfre, litere). Symbol (cyfra, litera) był dokładnie napisany* dobrze wprowadzona odpowiedź z klawiatury, jednak źle wybrany obraz symbolu (literka lub cyferka niedokładnie napisana).
 - *Chodziło o inny symbol (cyfre, litere). Symbol (cyfra, litera) był dobrze napisany* źle wprowadzona odpowiedź z klawiatury, obraz symbolu oraz symbol wprowadzony z klawiatury zgadzają się.
 - *Chodziło o inny symbol (cyfre, litere). Symbol (cyfra, litera) był niedokładnie napisany* napisany źle wprowadzona odpowiedź z klawiatury, obraz symbolu oraz symbol wprowadzony z klawiatury nie zgadzają się.
- Aby zostać w danej kategorii i nadal ćwiczyć (np. liczenie lub słowa), należy wcisnąć klawisz *Nowe zadanie*.
- Jeśli chcemy zmienić kategorię, wystarczy zaznaczyć interesujące nas działanie.
- Przy wyjściu z programu dostajemy podsumowanie dotychczasowych odpowiedzi - procent ile było poprawnych.



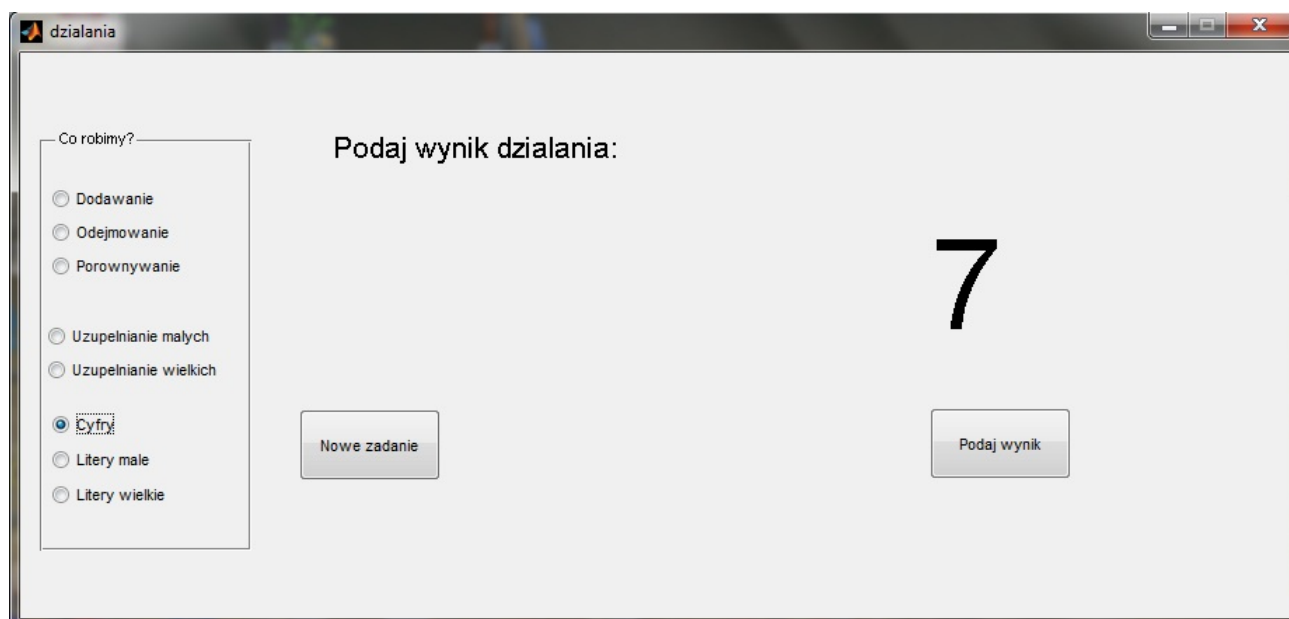
Rysunek 4.4: Działanie dodawania



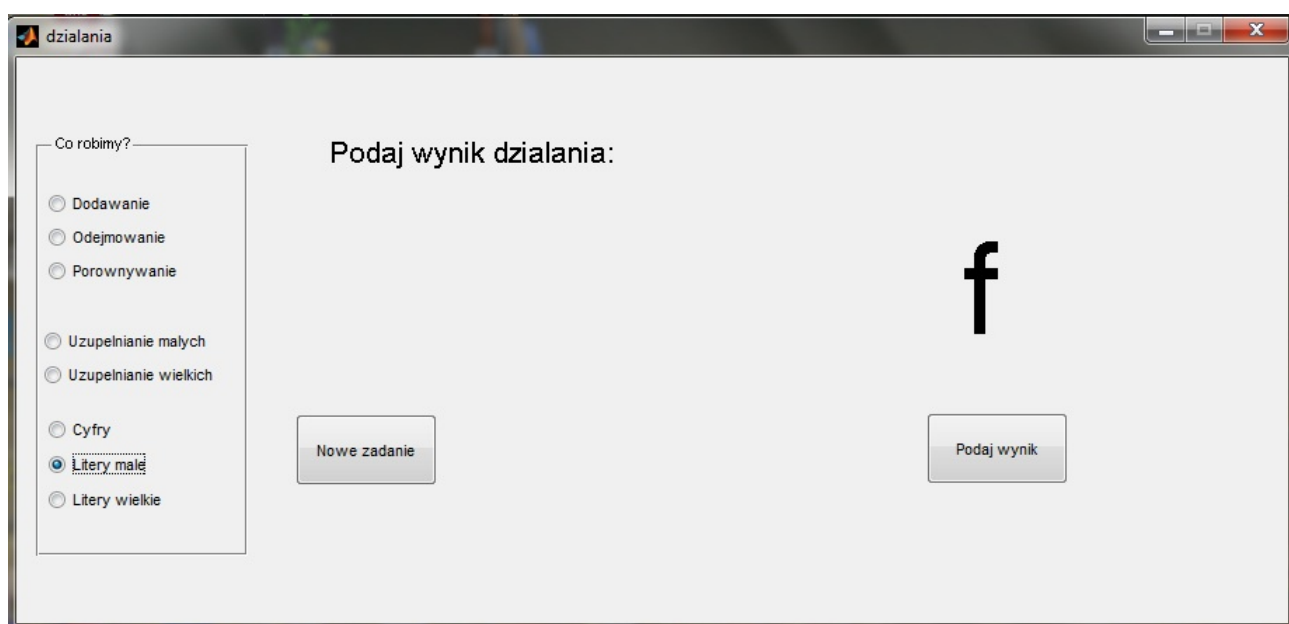
Rysunek 4.5: Porównywanie działań



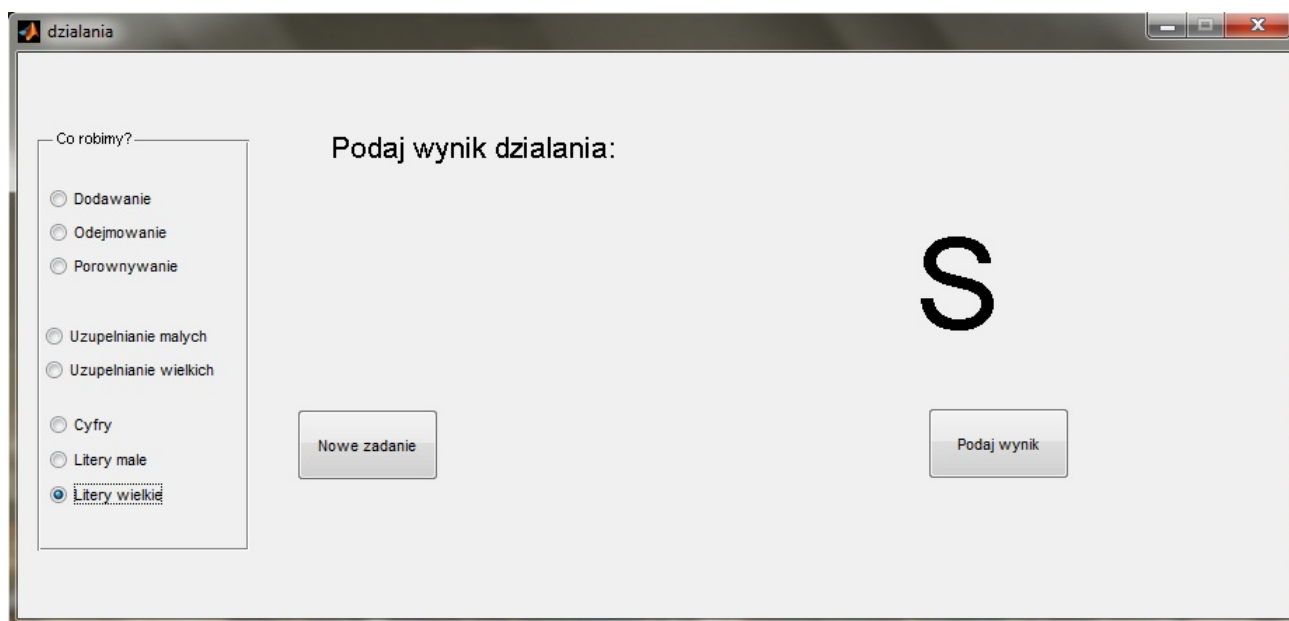
Rysunek 4.6: Uzupełnianie małych liter



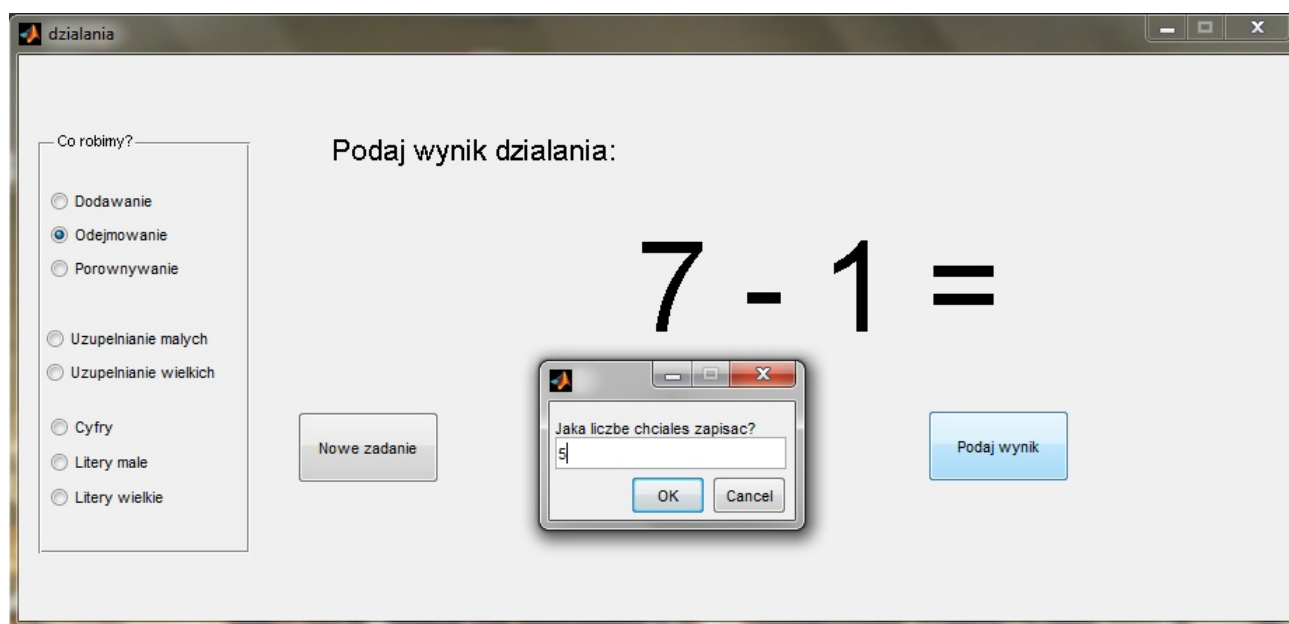
Rysunek 4.7: Pisanie cyfr



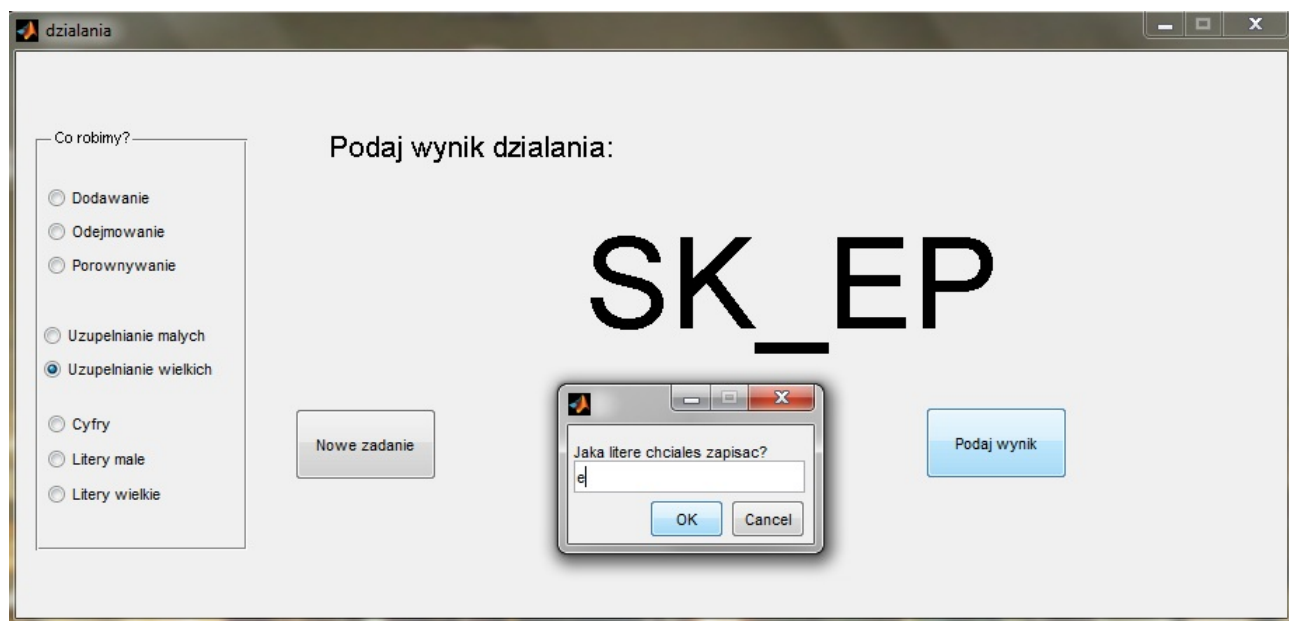
Rysunek 4.8: Pisanie małych liter



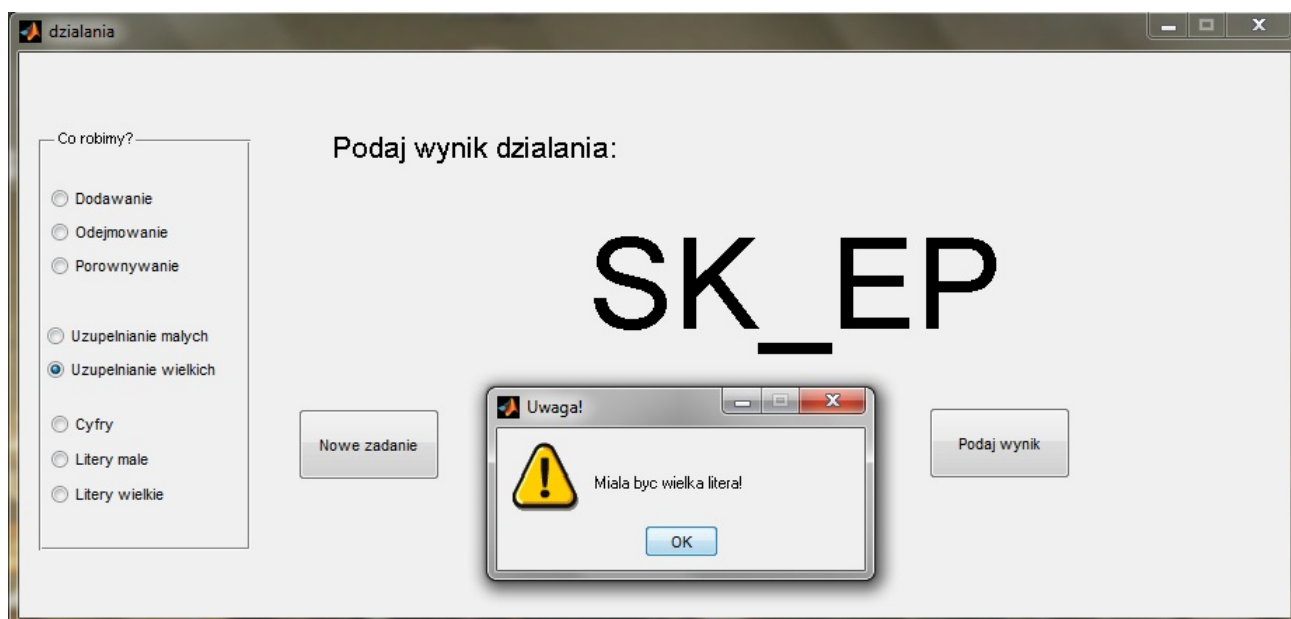
Rysunek 4.9: Pisanie wielkich liter



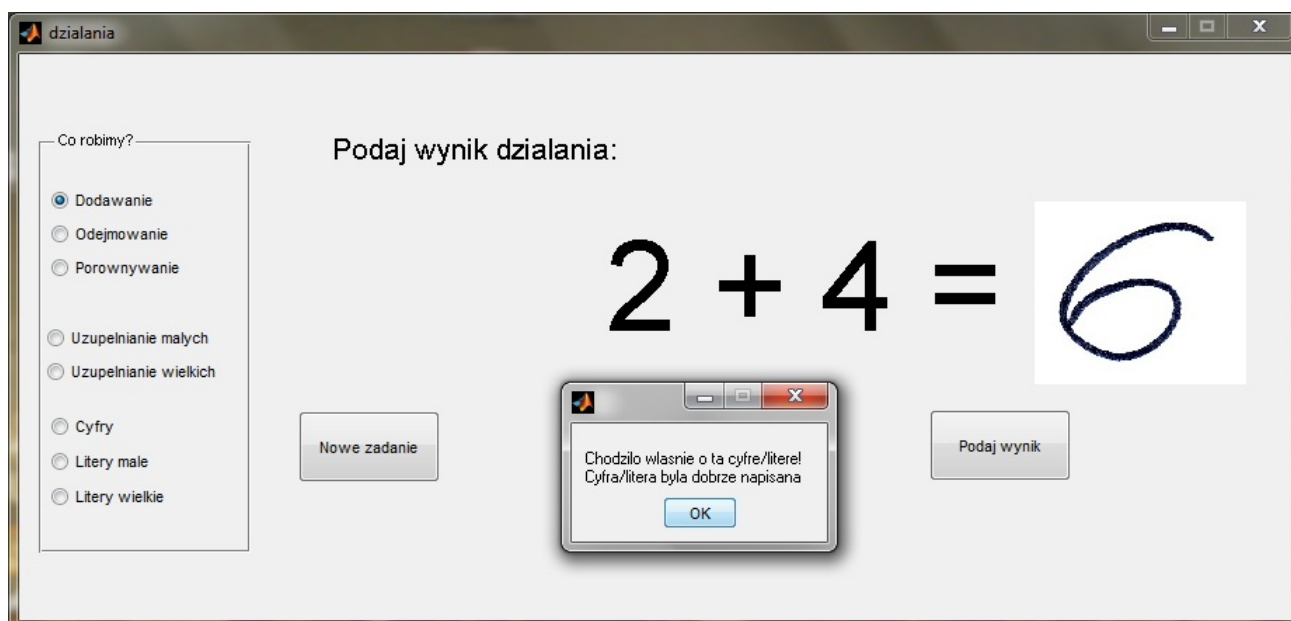
Rysunek 4.10: Wprowadzanie wyniku



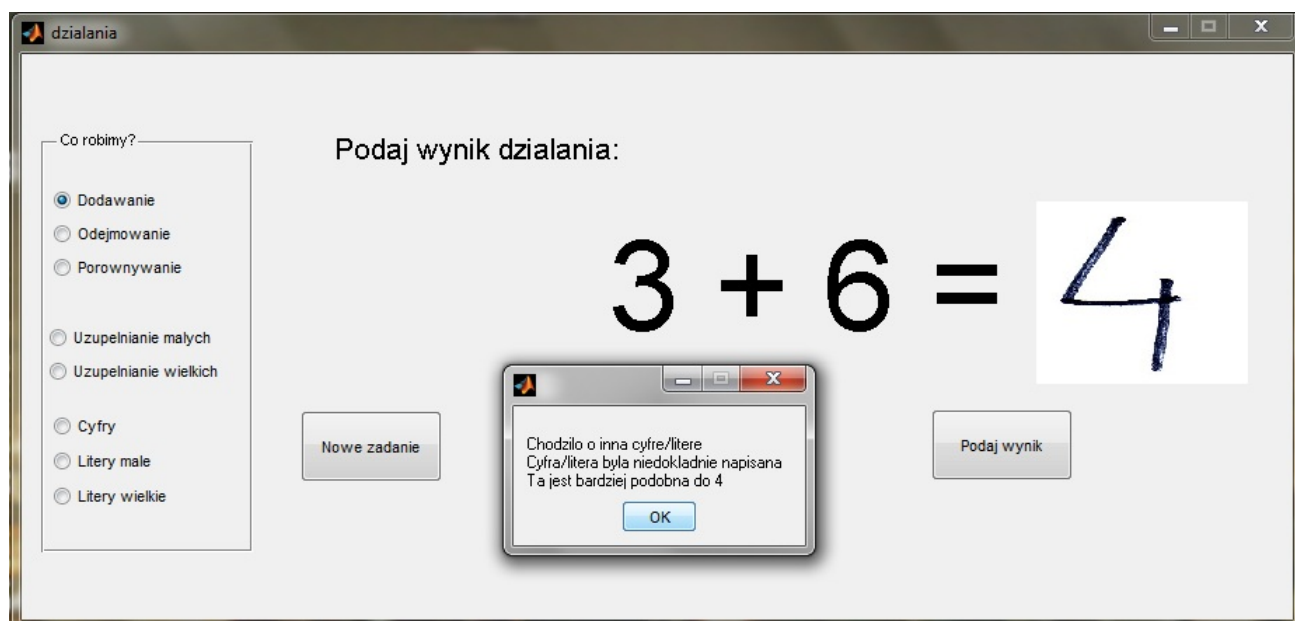
Rysunek 4.11: Uzupełnianie wielkich liter



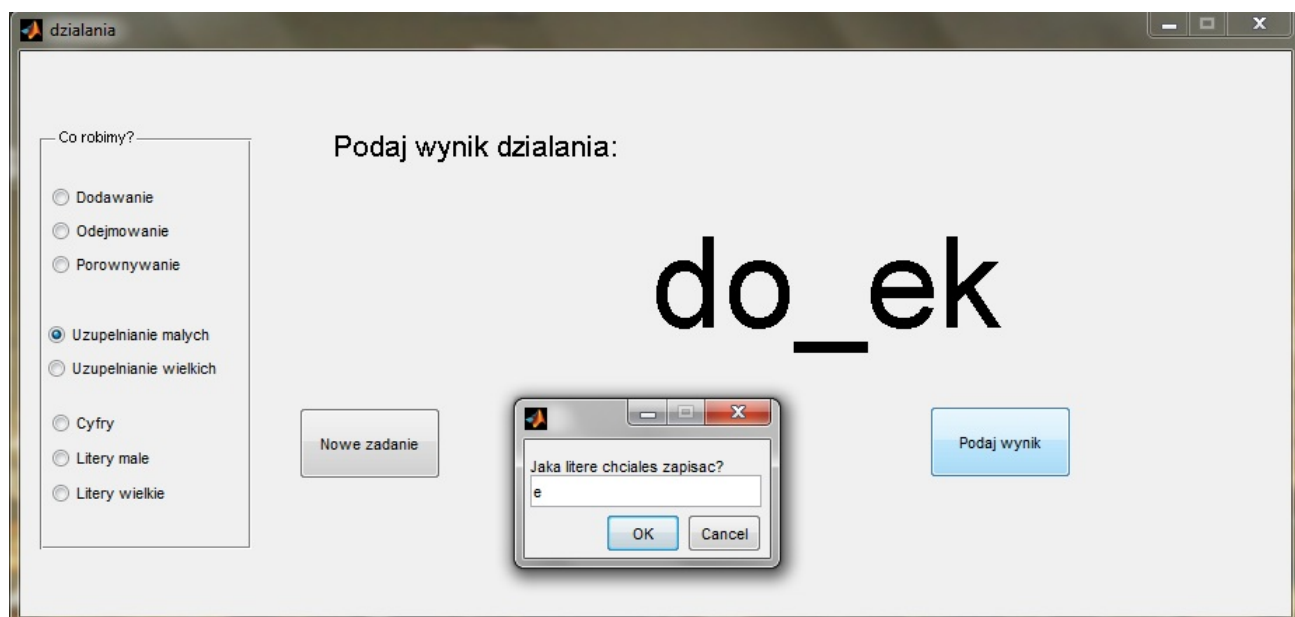
Rysunek 4.12: Błąd przy wprowadzaniu symbolu



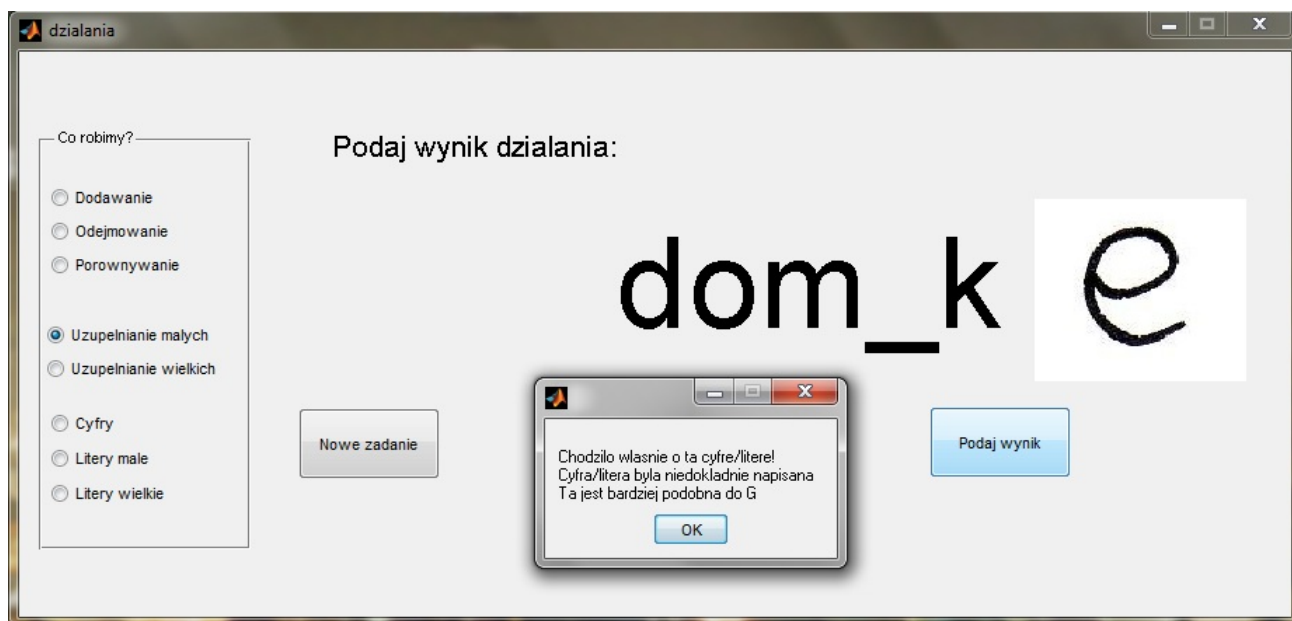
Rysunek 4.13: Dobry wynik, dobry obraz



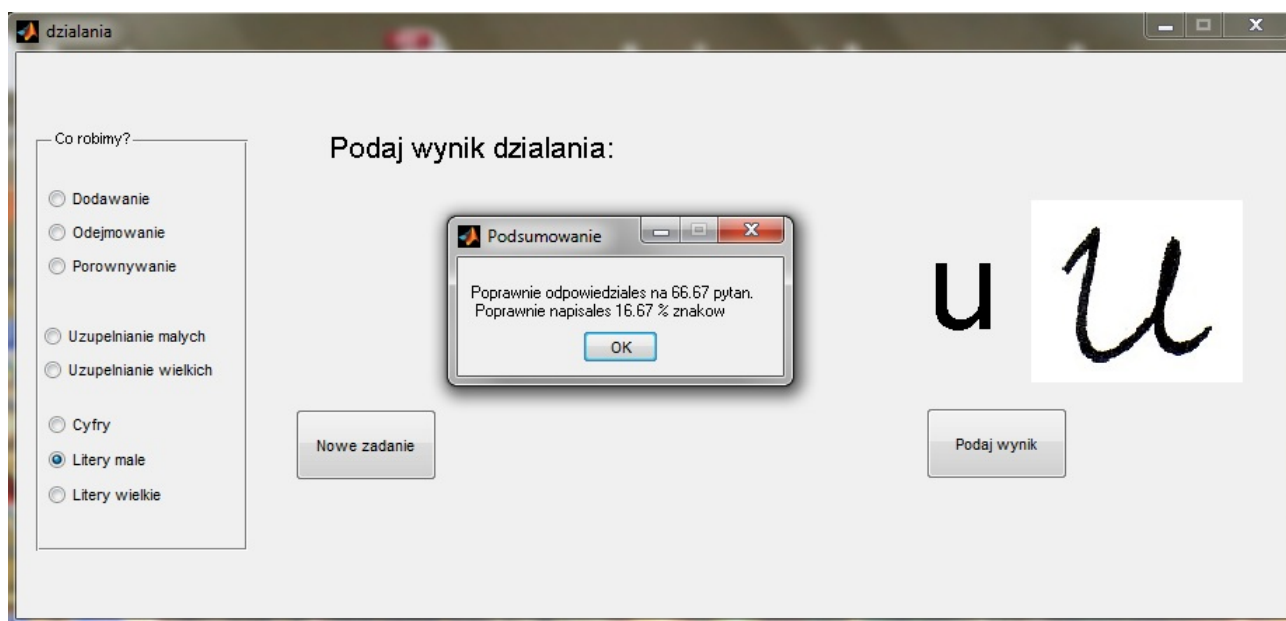
Rysunek 4.14: Niepoprawny wynik, cyfry się nie zgadzają



Rysunek 4.15: Uzupełnianie małych liter



Rysunek 4.16: Dobra odpowiedź, niedokładnie napisana



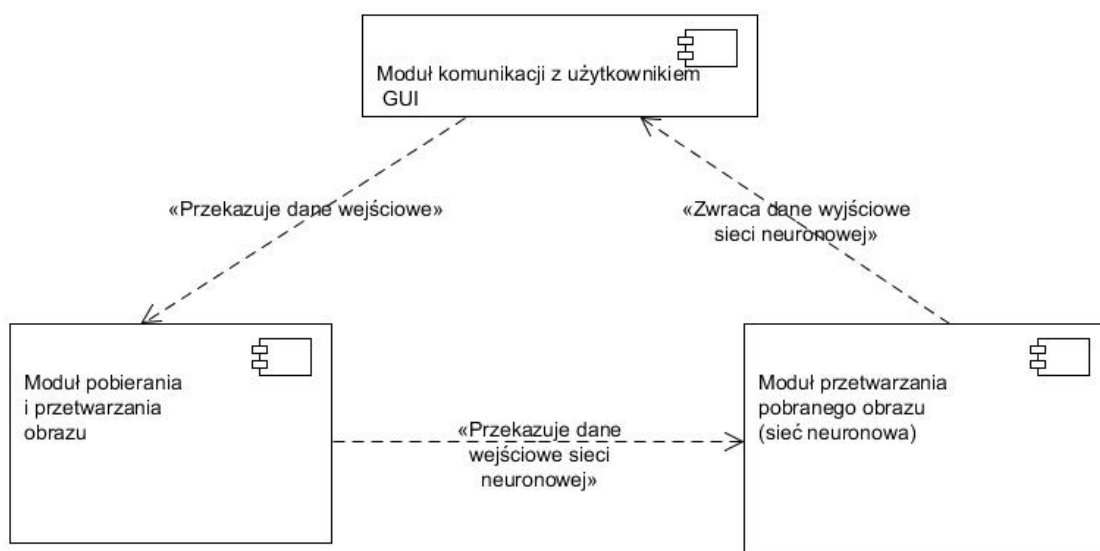
Rysunek 4.17: Podsumowanie

4.6. Opis modułów systemu

Realizacja projektu polega na rozwiązaniu dwóch zasadniczych problemów: przetwarzania obrazu oraz zastosowania sztucznych sieci neuronowych. W następnej kolejności omówię sposób przetwarzania danych oraz zastosowaną sieć neuronową.

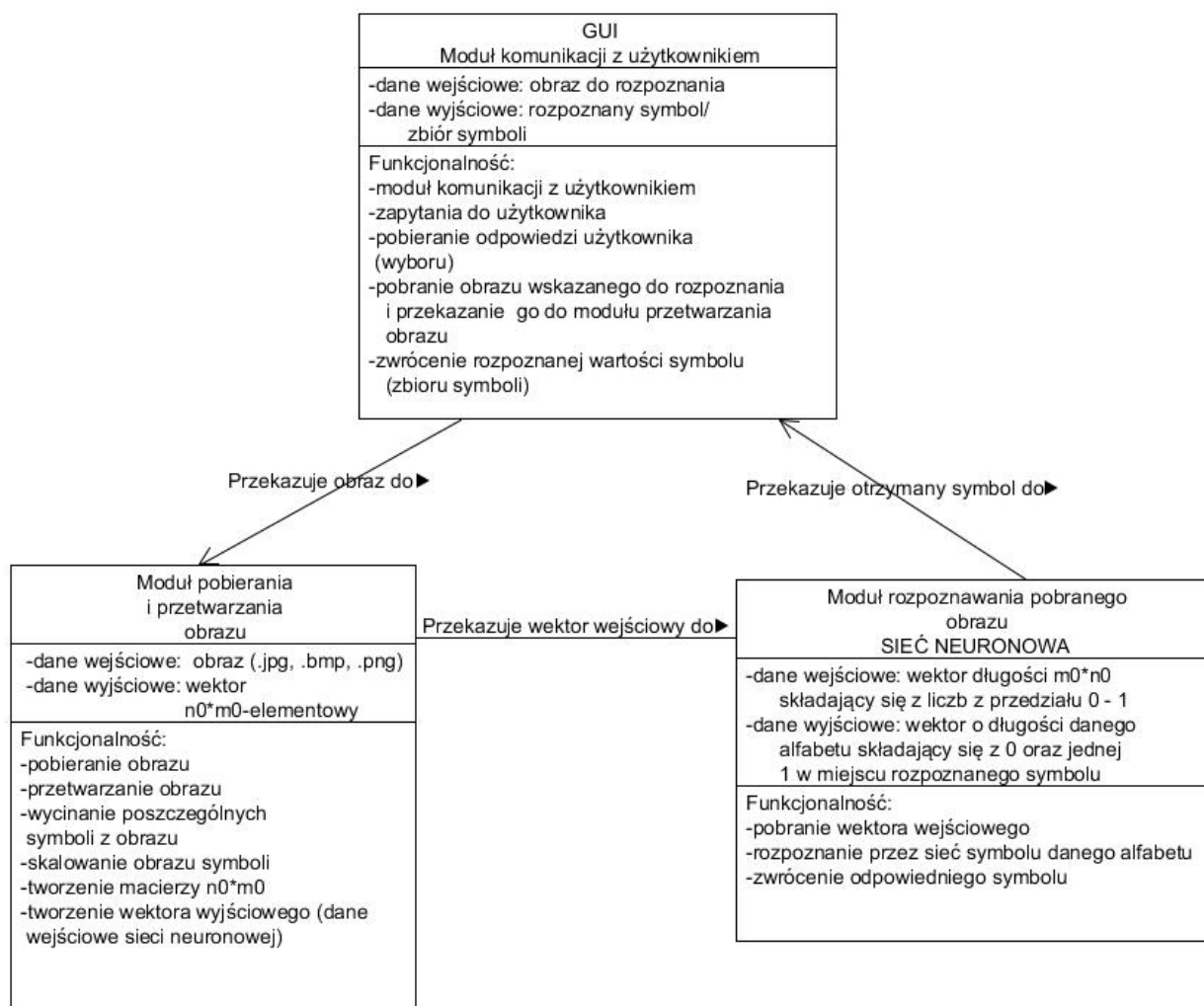
Projekt jest zrealizowany w Programie *MATLAB R2009b*.

Poniższy diagram przedstawia najważniejsze komponenty systemu.



Rysunek 4.18: Diagram komponentów

Szczegółowa struktura systemu OCR.



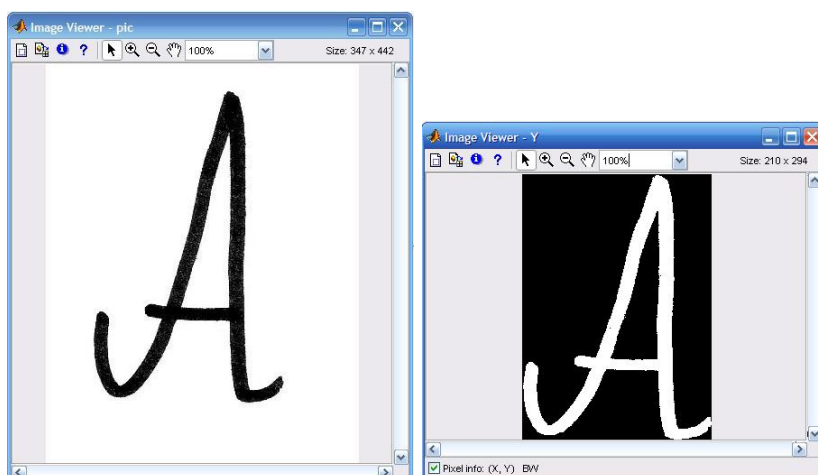
Rysunek 4.19: Struktura systemu OCR

4.6.1. Dokładny opis modułów systemu

Moduł pobierania oraz przetwarzania obrazu

- Na potrzeby programu stworzona zostaje funkcja, która pobiera jako parametr obrazek (.jpg, .bmp, .png), a zwraca macierz (*preproces.m*), następnie zamienia macierz na wektor (*wektor_uczacy.m*).
 - Obraz zostaje wczytany.
 - Z wczytanego obrazu tworzony jest jego obraz binarny (elementy macierzy to 0 i 1).
 - Obraz jest odwracany (kolor biały jest zamieniany na czarny i na odwrót, 1 zamieniane jest na 0, a 0 na 1).

- Zmieniany jest wymiar obrazu - szerokość jest wielokrotnością podanej liczby $m0$, a wysokość wielokrotnością liczby $n0$.
- Wielkości $n0$ i $m0$ podawane są w trakcie tworzenia sieci (budowanie sieci).
- Przekształcenie to zostaje dokonane za pomocą funkcji wbudowanej w MATLABIE (*imresize*).
- Ponowna zmiana wielkości obrazu - algorytm opisany poniżej:
 - * Obraz dzielony jest na $n0*m0$ pól, wielkość może być zmieniana.
 - * W każdym polu zliczana jest ilość pikseli.
 - * Pole z maksymalną ilością pikseli otrzymuje wartość 1.
 - * Wartości pól obliczane są poprzez podzielenie ilości pikseli przez maksymalną ilość.
- Otrzymana macierz ma wymiary $n0*m0$ (w zależności od wyboru) zawierająca liczby od 0 do 1.
- Jest to reprezentacja symbolu.
- Powstała macierz przekształcana jest do postaci wektora (najpierw pierwszy wiersz, potem drugi itd)- funkcja *wektor_uczacy.m*.
- W wyniku tych operacji otrzymany zostaje $n0*m0$ -elementowy wektor (ilość elementów może ulec zmianie w przypadku innych wymiarów otrzymanej macierzy).



Rysunek 4.20: Wczytana literka A oraz po przekształceniu.

W taki sposób pliki graficzne zostają doprowadzone do postaci w jakiej tworzone są literki w przykładzie OCR w Matlabie. Tak otrzymany wektor może być wprowadzony na wejście sieci neuronowej.

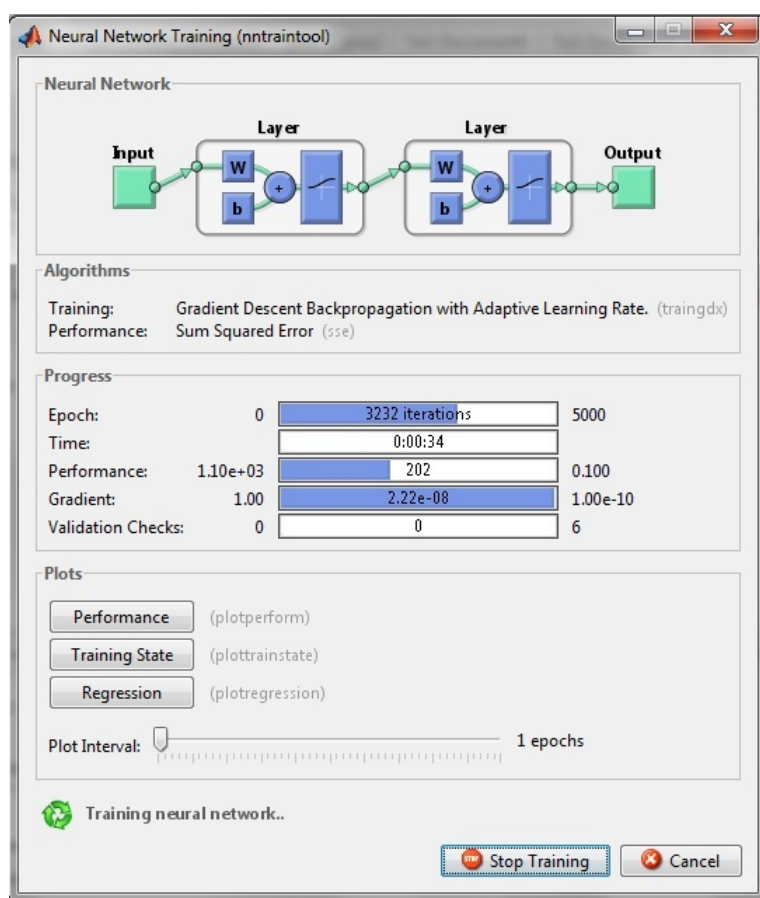
Sztuczna sieć neuronowa

W projekcie zastosowana została sztuczna sieć neuronowa przedstawiona w Matlabie. Sieć uczona jest metodą wstecznej propagacji błędów ze współczynnikiem momentum. Sieć neuronowa tworzona oraz trenowana jest w pliku *budowa_sieci.m* - opis tworzenia w instrukcji użytkownika

- `S2 = size(T,1);`
- `net = newff(minmax(P),[S1 S2], 'logsig' 'logsig', 'traingdx');`
Ilości neuronów, warstwa wyjściowa S2 musi mieć tyle ile jest możliwych odpowiedzi - dlatego jest to `size(T,1)`.
- `net.performFcn = 'sse';`
Metoda oceny wydajności sieci, wybrana jest *sse*, czyli suma kwadratów błędów (np. dla wymaganej odpowiedzi `[0 1 0]` a otrzymanej `[0.5 1 0]` będzie to $0.5^2 + 0^2 + 0^2 = 0.25$).
- `net.trainParam.goal = 0.1;`
Wartość wydajności, przy jakiej sieć uznamy za nauczoną (wydajność - patrz wyżej).
- `net.trainParam.show = 20;`
Co ile epok pokazuje wynik.
- `net.trainParam.epochs = 5000;`
Ile iteracji wykonać maksymalnie (liczba epok).
- `net.trainParam.mc = 0.95;`
Współczynnik momentum.
- `net = train(net,P,T);`

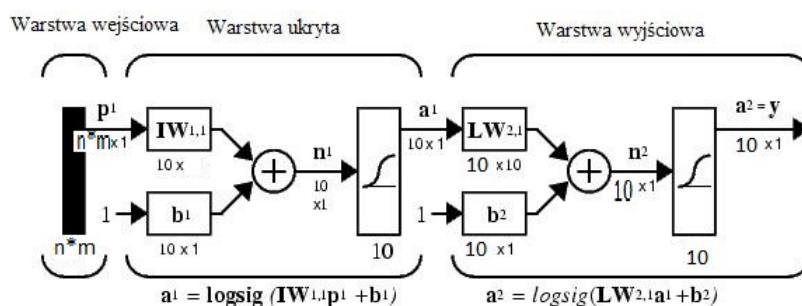
Sieć ta jest siecią nadzorowaną (użyto wektorów uczących), jednokierunkową nieliniową z logistyczną funkcją aktywacji oraz jedną warstwą ukrytą. Ilość neuronów w warstwie wejściowej uzależniona jest od tego, na ile elementów dzielony jest obraz ($n0*m0$) Otrzymany obraz (wycięty symbol) dzielony jest na $n0*m0$ mniejszych jednostek. Dlatego też w warstwie wejściowej jest $n0*m0$ neuronów. Odpowiada to wektorowi z $n0*m0$ elementami (sposób jego otrzymania został omówiony w poprzednim podrozdziale). Jeśli chcemy uzyskać większą dokładność i podzielimy obraz na więcej elementów, wtedy należy odpowiednio zwiększyć ilość neuronów w warstwie wejściowej (a niekiedy i w ukrytej). Wartości dla danych neuronów odpowiadają wartościom otrzymanym dla wektora wejściowego. Idealne byłyby to wartości 0 i 1, jednak biorąc pod uwagę szumy, wartości te zostają zmienione.

Ilość neuronów w warstwie ukrytej również determinowana jest w trakcie tworzenia sieci (budowa sieci). Dla sieci do rozpoznawania cyfr, wystarczająca ilość neuronów w warstwie ukrytej to 10, a dla liter 20. Liczba ta została oszacowana na podstawie wcześniejszych doświadczeń. Jeśli dana sieć ma problemy w nauce, można dodać neurony do warstwy ukrytej. Na wyjściu sieci w wersji podstawowej jest N neuronów (dla alfabetu N elementowego, czyli 10 neuronów dla cyfr, odpowiednio więcej neuronów dla sieci z małymi oraz wielkimi literami). Liczba ta odpowiada ilości symboli w danym alfabecie. Na wyjściu sieć zwraca więc N-elementowy wektor odpowiadający długości danego alfabetu (10 w przypadku cyfr). W miejscu rozpoznanego symbolu na wyjściu sieci jest 1, a w pozostałych 0. Np. jeśli sieć rozpoznała daną literę jako "A" to wektor wyjściowy ma wartość 1 na pierwszym miejscu, a na pozostałych 0. Dane wyjściowe uzasadniają dwukrotne użycie unipolarnej sigmoidalnej funkcji aktywacji. Jest to funkcja ciągła, więc przyjmuje czasami wartości pomiędzy 0 i 1 (z powodu szumów, trudno otrzymać idealne 0 i 1). Z tego powodu użyta zostaje funkcja *compete* w Matlabie. Funkcja ta sprawia, iż



Rysunek 4.21: Sieć neuronowa w trakcie nauki

wyjście odpowiadające danej literze (symbolowi), która najbardziej przypomina wektor wejściowy przyjmuje wartość 1, a pozostałe przyjmują wartości 0. Dopiero wynik tego procesu jest właściwym wyjściem sieci.



Rysunek 4.22: Sieć neuronowa użyta w programie do rozpoznawania cyfr.

W przypadku, gdy dany alfabet ma mniej lub więcej znaków (np. polski alfabet ma 32 litery, cyfry składają się z 10 różnych znaków) sieć ma odpowiednio więcej lub mniej neuronów w warstwie wyjściowej. W przypadku programu dla dzieci zbiór znaków został podzielony na 4 grupy:

- Małe litery.

- Duże litery.
- Cyfry.
- Znaki.

Sieć klasyfikująca wszystkie znaki (małe i duże litery oraz cyfry) musiałaby mieć ponad 70 neuronów w warstwie wyjściowej. W takim przypadku należałoby zwiększyć ilość neuronów w warstwie wejściowej (poprzez dokładniejsze czytanie obrazka, podział na więcej części, większą rozdzielczość przetwarzanego obrazu). Należałoby również odpowiednio zwiększyć ilość neuronów w warstwie ukrytej.

Moduł komunikacji z użytkownikiem - GUI

- Najważniejszym zadaniem modułu komunikacji z użytkownikiem jest pobieranie danych oraz zwracanie wyników sieci neuronowej.
- Moduł służy również do zadawania pytań oraz pobierania odpowiedzi użytkownika.
- W programie istnieją dwa moduły komunikacji z użytkownikiem - z użytkownikiem, który nadzoruje sieć (budowa oraz trenowanie sieci) *budowa sieci.m* oraz z użytkownikiem, który korzysta z sieci *dzialania.m*.
- Szczegółowy opis działania modułów komunikacji z użytkownikiem opisany jest w instrukcji obsługi programu.

Rozdział 5

Dokumentacja kodu

5.1. Pliki

Budowa sieci:

- *budowa_sieci.m*, *budowa_sieci.fig* – główna aplikacja.

Test:

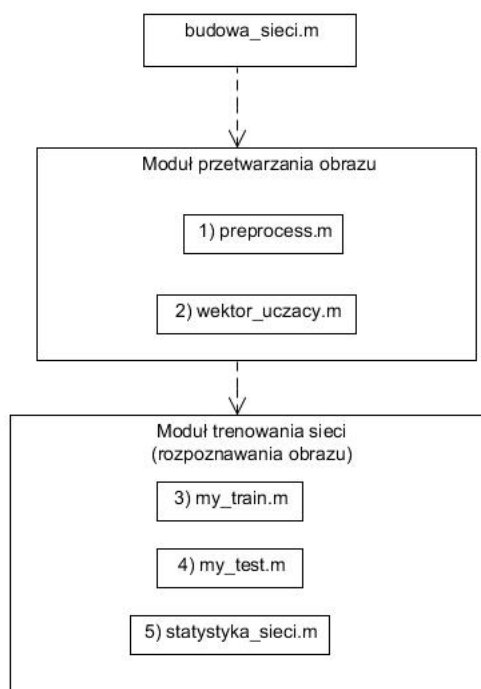
- *dzialania.m*, *dzialania.fig* – główna aplikacja,
- *generuj_dodawanie.m* – funkcja generująca działanie dodawania,
- *generuj_odejmowanie.m* – funkcja generująca działanie odejmowania,
- *generuj_porownanie.m* – funkcja generująca porównanie wyników dwóch działań,
- *generuj_slowo.m* – generacja słowa z brakującą literą,
- *my_check.m* – sprawdzenie odpowiedzi sieci na zadany rysunek,
- *my_test.m* – generacja odpowiedzi sieci dla wszystkich plików w danym folderze,
- *my_train.m* – trening sieci na danych w zdefiniowanym folderze,
- *preprocess.m* – przygotowanie pliku – z rysunku do macierzy o wymiarach **n x m**,
- *statystyka_sieci.m* – wygenerowanie statystyk sieci (ilość odpowiedzi),
- *wektor_uczacy.m* – budowa wektora uczącego na podstawie danych z preprocessingu.

5.2. Dokładny opis plików aplikacji

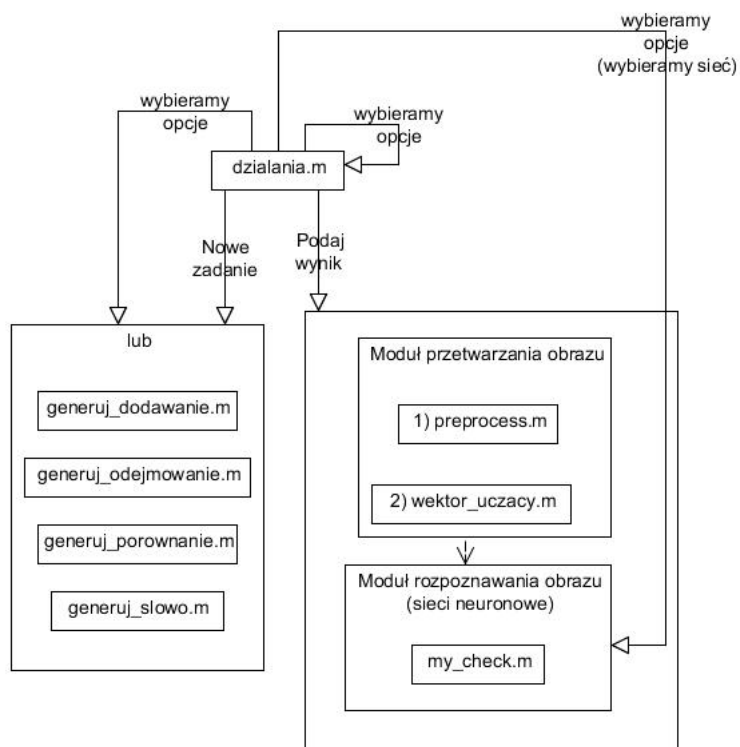
5.2.1. *dzialania.m* / *dzialania.fig*

Plik zawierający konstrukcję okna (*dzialania.fig*) oraz obsługę zdarzeń (*dzialania.m*)

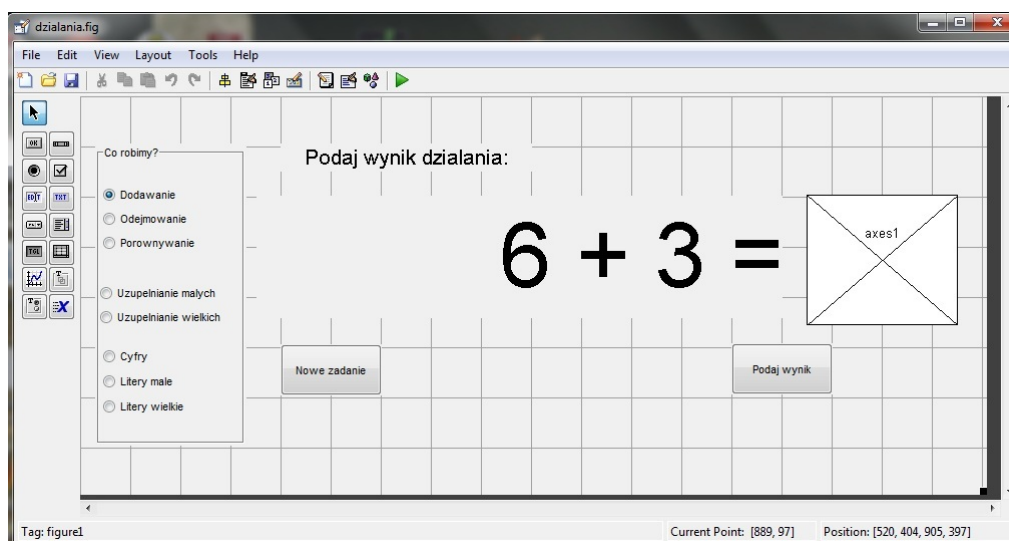
Podstawowe elementy (kontrolki aplikacji) to (patrząc od lewej strony i od góry):



Rysunek 5.1: Główna aplikacja do budowania oraz trenowania sieci.



Rysunek 5.2: Główna aplikacja do rozpoznawania symboli



Rysunek 5.3: Okno programu.

- panel z wyborem działania *Co robimy?* (uipanel1),
- tekst z treścią zadania (text2),
- przycisk generacji nowego zadania *Nowe zadanie* (pushbutton2),
- okno prezentacji wyniku (axes1),
- przycisk wprowadzania wyniku *Podaj wynik* (pushbutton1).

Funkcje obsługujące zdarzenia zostaną opisane poniżej.

text2 – tworzenie pola (funkcja inicjalizująca)

```
function text2_CreateFcn(hObject, eventdata, handles)
```

Funkcja ta wywoływana jest jednokrotnie przy uruchomieniu programu.

Algorytm:

1. Wczytanie pliku *cyfry.mat* zawierającego sieć do rozpoznawania cyfr.
2. Zapis sieci do struktury handles (*num.net*).
3. Wczytanie pliku *littery_m* zawierającego sieć do rozpoznawania małych liter.
4. Zapis sieci do struktury handles (*littery_m.net*).
5. Wczytanie pliku *littery_w* zawierającego sieć do rozpoznawania wielkich liter.
6. Zapis sieci do struktury handles (*littery_w.net*).
7. Generacja pierwszego zadania – *generuj_dodawanie()* – rozdział 5.2.9.
8. Wczytanie słów wykorzystywanych do nauki.
9. Wyświetlenie pierwszego zadania.
10. Ustawienie wartości początkowych elementom struktury handles (wybor=1, poprawne=0, odpowiedzi=0).

Nowe zadanie – wciśnięcie przycisku

```
function pushbutton2_Callback
```

Algorytm:

1. Sprawdzenie, które zadanie jest wybrane w panelu *Co robimy?*.
 - (a) *Dodawanie* - generowane jest działanie dodawania – funkcja *generuj_dodawanie()* – rozdział 5.2.9.
 - (b) *Odejmowanie* - generowane jest działanie odejmowania – funkcja *generuj_odejmowanie()* – rozdział 5.2.10.
 - (c) *Porównywanie* - generowane jest porównywanie wyników działań – funkcja *generuj_porownanie()* – rozdział 5.2.11.
 - (d) *Uzupełnianie_malych\duzych* - generowane jest zadanie z uzupełnianiem liter (małych, bądź dużych) – funkcja *generuj_slowo()* – rozdział 5.2.12.
 - (e) *Cyfry* – losowana jest cyfra – funkcja *randi()*.
 - (f) *Litery male\wielkie* – losowana jest litera mała, bądź wielka – funkcja *randi()*.
2. Wyświetlany jest nowy tekst zadania w polu *text2*.
3. Czyszczone jest okno wyniku *axes1*.

Co robimy? – wybór innej opcji

```
function uipanel1_SelectionChangeFcn(hObject, eventdata, handles)
```

Algorytm:

1. Sprawdzany jest aktualny wybór.
2. Wybór zapisywany zostaje do struktury *handles* (pole *wybor*).
3. Wywoływana jest funkcja generacji nowego zadania *pushbutton2_Callback()* – patrz wyżej.

Podaj wynik – wciśnięcie przycisku.

```
function pushbutton1_Callback
```

Algorytm:

1. Otwarcie okna *inputdlg()* z zapytaniem o liczbę\literę jaką chciało się zapisać.
2. Sprawdzenie, czy przy prośbie o literę podano literę, lub w prośbie o cyfrę podano cyfrę.
 - (a) Przy błędnym podaniu (litera\cyfra) wyświetlany jest komunikat i następuje powrót do punktu 1.
 - (b) Przy prawidłowym podaniu, przechodzi się do punktu 3.
3. Otwarcie okna z wyborem pliku graficznego – *uigetfile('*.png;*.bmp;*.jpg')* i pobranie pliku z rysunkiem.

4. Wyświetlenie rysunku – *imagesc()*
5. Wybór odpowiedniej sieci w zależności od tego, czy do rozpoznania jest wielka, mała litera, cyfra, czy znak (sieci zapisane są w strukturze handles).
6. Obliczenie odpowiedzi sieci – *my_check(net,filename)* – rozdział 5.2.5.
7. Zamiana wartości liczbowej z funkcji *my_check()* na liczbę\literę poprzez wykorzystanie zmiennej *prefix_c* (ze struktury handles).
8. Porównanie odpowiedzi sieci z odpowiedzią prawidłową i odpowiedzią wpisaną do okna w punkcie 1.
9. Wyświetlenie informacji na temat wyniku (poprawnie\niepoprawnie).
10. Zapisanie wyniku w strukturze handles (pola poprawne i odpowiedzi).

Wysłanie zadania zamknięcia aplikacji

`function figure1_CloseRequestFcn`

Funkcja wywoływana jest po wciśnięciu przycisku *zamknij*, lub np. wciśnięciu kombinacji Alt+F4. **Algorytm:**

1. Obliczenie ilości poprawnych odpowiedzi na podstawie pomiarów zapisanych w strukturze handles (odsetek = poprawne/odpowiedzi).
2. Wyświetlenie okna z informacją o ilości poprawnych odpowiedzi i oczekiwanie na zamknięcie tego okna – *uiwait()*.
3. Zamknięcie głównego okna programu.

5.2.2. *budowa_sieci.m* / *budowa_sieci.fig*

Funkcje opisujące GUI tworzące sieć na podstawie rysunków z danego folderu (dwa foldery powinny być dostępne – z danymi uczącymi i testującymi)

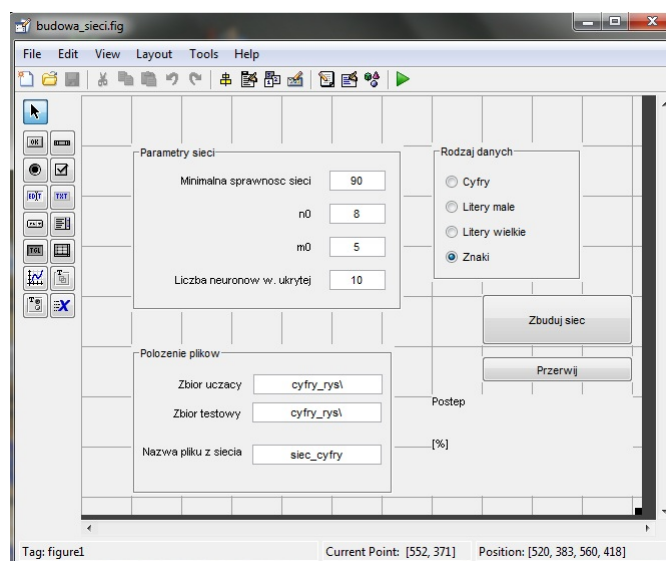
W pliku *budowa_sieci.m* wykorzystywana jest tylko jedna funkcja – obsługa przycisku *Zbuduj siec*.

`function pushbutton1_Callback`

Algorytm:

1. Pobranie informacji na temat rodzaju sieci – panel Rodzaj danych.
2. Budowa tablic *prefix* i *prefix_c* w zależności od rodzaju sieci. *prefix* zawiera informacje jak nazywają się pliki (początek nazwy dla danej klasy zawsze musi być taki sam), a *prefix_c* – jaką klasę odpowiedni *prefix* reprezentuje, np.


```
prefix = {'zero','jeden','dwa','trzy','cztery','piec','szesc','siedem',...
          'osiem','dziewiec'};
prefix_c = '0123456789';
```
3. Pobranie nazw folderów do nauki i testu (panel *Polozenie plikow*).
4. Pobranie nazwy pliku **.mat* pod jaką ma zostać zapisana sieć.



Rysunek 5.4: Główne okno programu.

5. Pobranie parametrów obrazu do nauki – rozmiar $n0 \times m0$.
6. Pobranie liczby neuronów w warstwie ukrytej.
7. Pobranie minimalnej sprawności sieci, która zostanie uznana za zadowalającą.
8. Przeprowadzenie treningu sieci o zadanych parametrach na podstawie rysunków z folderu nauki.

```
net = my_train(dirname,prefix,licza_n_wu);
```

- rozdział 5.2.3.

9. Przeprowadzenie testu sieci na podstawie rysunków z folderu testu.

```
[Y,Tx] = my_test(net,test_dirname,prefix);
```

- rozdział 5.2.4.

10. Obliczenie sprawności sieci na podstawie wyników testu.
11. Wyświetlenie informacji na temat sprawności sieci.
12. Jeżeli sprawność mniejsza od wymaganej, powrót do punktu 8.
13. Zapis zbudowanej sieci do mat-pliku o podanej nazwie.
14. Wyświetlenie informacji o utworzeniu sieci.

5.2.3. *my_train.m*

Trening sieci na podstawie rysunków z danego folderu.

```
function net=my_train(dirname,prefix,liczba_n_WU)
```

Wejście:

- *dirname* – nazwa katalogu z danymi uczącymi.

- *prefix* – początki nazw rysunków należących do kolejnych klas np.
{‘zero_’, ‘jeden_’, ...}.
- *liczba_n_WU* – liczba neuronów w warstwie ukrytej.

Wyjście:

- *net* – zbudowana sieć.

Algorytm:

1. Wyszukanie plików w danym folderze o zadanych prefiksach – *dir()*.
2. Budowa wektorów uczących:
 - (a) Odczyt każdego z plików.
 - (b) Preprocessing – rozdział 5.2.7.
 - (c) Budowa wektora uczącego na podstawie rysunku – rozdział 5.2.8.
 - (d) Zapis wektora uczącego do macierzy P.
 - (e) Zapis klasy do której należy rysunek do macierzy T.

3. Budowa sieci typu feed-forward

```
net = newff(minmax(P),[liczba_n_WU, size(T,1)],{'logsig' 'logsig'},'traingdx');
```

4. Ustawienie parametrów nauki

- (a) `perform_Fcn = 'sse'` – obliczenie wydajności metodą najmniejszych kwadratów,
- (b) `trainParam.goal = 0.1` – wymagana wydajność sieci,
- (c) `trainParam.epochs = 500` – maksymalna liczba epok

5. Trening sieci

```
net = train(net,P,T);
```

5.2.4. *my_test.m*

Przeprowadzenie testu sieci – klasyfikacja wszystkich rysunków z danego folderu.

```
function [Y,T]=my_test(net,dirname,prefix)
```

Wejście:

- *net* – sieć do przetestowania (np. wyjściowa z funkcji *my_train()* – rozdział 5.2.3).
- *dirname* – nazwa folderu z rysunkami do testów.
- *prefix* – początki nazw rysunków należących do kolejnych klas np.
{‘zero_’, ‘jeden_’, ...}.

Wyjście:

- *Y* – wyniki klasyfikacji poszczególnych rysunków.
- *T* – prawidłowe odpowiedzi

Algorytm:

1. Wczytanie plików z danego folderu dla każdego z prefixów.
2. Budowa wektorów uczących – identycznie jak w rozdziale 5.2.3.
3. Test sieci – $\text{compet}(\text{sim}(\text{net}, P))$.

5.2.5. *my_check.m*

Test działania sieci dla danego pliku.

```
function y = my_check(net,filename)
```

Wejście:

- *net* – sieć.
- *filename* – nazwa pliku do przetestowania.

Wyjście:

- *y* – odpowiedź sieci – gdzie zaklasyfikowano dany plik.

Algorytm:

1. Wczytaj dany rysunek – $\text{imread}(\text{filename})$.
2. Przeprowadź wstępną obróbkę – $\text{preprocess}()$ – rozdział 5.2.7.
3. Zbuduj wektor uczący – $T1 = \text{wektor_uczacy}()$ – rozdział 5.2.8.
4. Przetestuj sieć dla zadanego wektora
$$y = \text{compet}(\text{sim}(\text{net}, T1))$$

5.2.6. *statystyka_sieci.m*

Budowa pliku ze statystyką sieci – rzeczywiste klasy vs klasy wg. sieci.

```
function statystyka_sieci(Y,T,prefix,filename)
```

Wejście:

- *Y* – odpowiedzi sieci.
- *T* – prawidłowe odpowiedzi.
- *prefix* – nazwy klas, np. '0', '1', '2' ...
- *filename* – nazwa pliku w którym zapisany zostanie raport ze statystyką.

Wyjście:

- Brak (bezpośredni zapis wyników do pliku).

Algorytm:

1. Dla każdej z klas:

- (a) Znalezienie w wektorze T miejsc wystąpienia danej klasy (kiedy przykładowo odpowiedź powinna wynosić 1).
 - (b) Wycięcie z wektora Y elementów, znalezionych w podpunkcie a (czyli przykładowo, wycięcie elementów, gdzie odpowiedź powinna wynosić '1').
 - (c) Dla każdej z klas obliczenie ile jest elementów w wektorze b (czyli przykładowo, ile jest odpowiedzi '0', ile jest odpowiedzi '1' itd.) i przeliczenie wartości na procenty (podzielenie przez całkowitą liczbę odpowiedzi).
2. Utworzenie i otwarcie pliku.
 3. Dla każdej z klas zapis do pliku linii z wartościami % klasyfikacji każdej z klas.
 4. Zamknięcie pliku.

Przykładowy plik wyników:

zero	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
jeden	0.00	100.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
dwa	13.51	35.14	0.00	10.81	0.00	5.41	24.32	10.81	0.00	0.00
...										
dziewiec	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00

5.2.7. *preprocess.m*

Przeprowadzenie preprocessingu (orbóki wstępnej) na obrazie wejściowym (filtracja, zmiana rozdzielczości).

```
function I2 = preprocess(I0);
```

Wejście:

- Obraz $I0$ o dowolnych rozmiarach, kolorowy RGB bądź czarno-biały.

Wyjście:

- Macierz prezentująca obraz o rozmiarach $n0 \times m0$. Suma jej elementów wynosi 1.

Algorytm:

1. Dla obrazu kolorowego konwersja do czarno-białego.
2. Binaryzacja.
3. Operacja morfologiczna - otwarcie.
4. Negacja obrazu.
5. Wycięcie fragmentu na którym jest element (obcięcie pustych brzegów).
6. Podział obrazu na $n0 \times m0$ segmentów.
7. Policzenie ilości pikseli o wartości = 1 w każdym segmencie i zapis ich do macierzy $I2$.
8. Podzielenie każdego z elementów macierzy $I2$ przez sumę elementów w całości macierzy (w wyniku suma wszystkich elementów będzie wynosić 1).

5.2.8. *wektor_uczacy.m*

Budowa wektora uczącego na podstawie rysunku po preprocessingu.

```
function v = wektor_uczacy(I)
```

Wejście:

- I – obraz po preprocessingu.

Wyjście:

- v – wektor uczący – wejście do sieci.

Algorytm:

1. Zamień macierz (rysunek) na wektor $v=I(:);$.

5.2.9. *generuj_dodawanie.m*

Generacja działania dodawania.

```
function [str,score] = generuj_dodawanie()
```

Wejście:

- Brak

Wyjście:

- str – łańcuch postaci „a+b”.
- $score$ – oczekiwany wynik.

Algorytm:

1. Losuj pierwszą liczbę całkowitą $k1$ (w zakresie 1..8) – $randi(8)$.
2. Losuj drugą liczbę całkowitą $k2$ (w zakresie 1..9- $k1$).
3. Zbuduj łańcuch opisujący działanie dodawania

```
str = sprintf('%d + %d =',k1,k2);
```

4. Oblicz wynik –

```
score = num2str(k1+k2)
```

– wynik w postaci znaku.

5.2.10. *generuj_odejmowanie.m*

Generacja działania odejmowania.

```
function [str,score] = generuj_odejmowanie()
```

Wejście:

- Brak

Wyjście:

- *str* – łańcuch postaci „a-b=”.
- *score* – oczekiwany wynik.

Algorytm:

1. Losuj pierwszą liczbę całkowitą *k1* (w zakresie 1..8) – *randi(8)*.
2. Losuj drugą liczbę całkowitą *k2* (w zakresie 1..9-*k1*).
3. Zbuduj łańcuch opisujący działanie dodawania

```
str = sprintf('%d - %d =',max([k1,k2]),min([k1,k2]));
```
4. Oblicz wynik –

```
score = num2str(max([k1,k2])-min([k1,k2]));
```

– wynik w postaci znaku.

5.2.11. generuj_porownianie.m

Generacja porównania wyniku dwóch działań.

```
function [str,score] = generuj_porownianie()
```

Wejście:

- Brak

Wyjście:

- *str* – łańcuch postaci „a+b c+d”.
- *score* – oczekiwany wynik.

Algorytm:

1. Losuj pierwszą liczbę całkowitą *k1* (w zakresie 1..8) – *randi(8)*.
2. Losuj drugą liczbę całkowitą *k2* (w zakresie 1..9-*k1*).
3. Losuj trzecią liczbę całkowitą *k3* (w zakresie 1..8) – *randi(8)*.
4. Losuj czwartą liczbę całkowitą *k4* (w zakresie 1..9-*k3*).
5. Zbuduj łańcuch opisujący działania

```
str = sprintf('%d + %d _ %d + %d',k1,k2,k3,k4);
```
6. Oblicz wynik –

```
diff = (k1+k2)-(k3+k4);
```

```
if(diff<0)
```

```
score = '<';
```

```
elseif(diff==0)
```

```
score = '=';
else
score = '>';
end
```

– wynik w postaci znaku.

5.2.12. *generuj_slowo.m*

Genreacja zadania polegającego na uzupełnieniu słowa z brakującą literą, np. SAMOC_ÓD.

```
function [str,brak] = generuj_slowo(slowa,duze)
```

Wejście:

- *slowa* – struktura zawierająca wszystkie słowa do testów (np. wcześniej wczytane z pliku).
- *duze* – czy słowo ma być z małych (0), czy wielkich liter (1) .

Wyjście:

- *str* – łańcuch z brakującym znakiem, np. „SAMOC_ÓD”.
- *brak* – brakująca litera, np. „H”.

Algorytm:

1. Wylosowanie jednego ze słów ze zbioru.
2. Wylosowanie pozycji litery, która zostanie wycięta (od drugiej do przedostatniej).
3. Jeżeli słowo ma być z małych liter, zamiana wszystkich liter na małe – *lower()*, jeżeli z wielkich, na wielkie – *Upper()*.
4. Przypisanie brakującego znaku zmiennej *brak*.
5. Zamiana jednej z liter (brakującej) w słowie na znak „_” i zapis słowa z luką w zmiennej *str*.

Rozdział 6

Raport z przeprowadzanych testów

6.1. Testy jednostkowe

W moim przypadku testy jednostkowe polegały na sprawdzaniu działania poszczególnych modułów programu. Osobno sprawdzony został moduł wczytywania obrazu, przetwarzania obrazu, a na samym końcu jego rozpoznawania.

6.2. Testy integracyjne

Testy integracyjne polegały na sprawdzeniu, czy poszczególne moduły właściwie ze sobą współpracują, czy przekazywane dane mają wymagany format (np. dane wyjściowe przetwarzania wstępnego są danymi wejściowymi sieci neuronowej, więc muszą mieć odpowiedni format).

6.3. Testy akceptacyjne

Testy akceptacyjne polegały na sprawdzeniu działania całego systemu, nie tylko na tym czy działa, lecz również jak działa (np. sprawność sieci neuronowej, reakcja na złe, niepoprawne dane).

Program nie pozwala dziecku popełnić błędu przypisania znaku nienależącego do danego alfabetu (np. jeśli dziecko zaznaczy małe litery i wpisze z klawiatury wielką literę, bądź znak, bądź cyfrę - pojawi się komunikat o niepoprawnym symbolu).

Testy zostały przeprowadzone dla różnych parametrów sieci neuronowej.

6.4. Testy sieci neuronowej

Testy zapisywane są przy tworzeniu nowej sieci neuronowej w pliku tekstowym (o takiej samej nazwie, jak nazwa sieci, nadana podczas jej budowy). Parametry sieci:

- $S2 = \text{size}(T,1);$

- `net = newff(minmax(P),[S1 S2], 'logsig' 'logsig', 'traingdx');`
- `net.performFcn = 'sse';`
- `net.trainParam.goal = 0.1;`
- `net.trainParam.show = 20;`
- `net.trainParam.epochs = 5000;`
- `net.trainParam.mc = 0.95;`
- `net = train(net,P,T);`

Testowana sieć dla cyfr.

- Minimalna sprawność sieci 90
- `no = 8`
- `m0 = 5`
- Liczba neuronów warstwy ukrytej 10
- Zbiór uczący `male_train` (50 obrazków dla każdej cyfry)
- Zbiór testowy `male_test` (30 obrazków dla każdej cyfry)

Sprawność sieci 98.88%.

W poniższej tabeli kolejne wiersze, to kolejne liczby, a kolumny mówią ile procent odpowiedzi było na jaki numer, czyli patrząc na wiersz *siedem* można stwierdzić, że w 6.67% przypadków sieć stwierdziła, że to *jeden* a w pozostałych 93.33%, że to *siedem*.

Cyfra->	0	1	2	3	4	5	6	7	8	9
zero	100	0	0	0	0	0	0	0	0	0
jeden	0	100	0	0	0	0	0	0	0	0
dwa	0	0	100	0	0	0	0	0	0	0
trzy	0	0	0	95.45	0	4.55	0	0	0	0
cztery	0	0	0	0	100	0	0	0	0	0
piec	0	0	0	0	0	100	0	0	0	0
szesc	0	0	0	0	0	0	100	0	0	0
siedem	0	6.67	0	0	0	0	0	93.33	0	0
osiem	0	0	0	0	0	0	0	0	100	0
dziewiec	0	0	0	0	0	0	0	0	0	100

Testowana sieć dla wielkich liter.

- Minimalna sprawność sieci 30
- `no = 8`
- `m0 = 5`
- Liczba neuronów warstwy ukrytej 15
- Zbiór uczący `wielkie_train` (80 obrazków dla każdej litery)
- Zbiór testowy `wielkie_test` (5 obrazków dla każdej litery)

Sprawność sieci 32.69%.

W poniższej tabeli kolejne wiersze, to kolejne LITERY, a kolumny mówią ile procent odpowiedzi było na jaką literę.

Litera->	A	B	C	D	E	F	G	H	I	J	K
A_	16.25	0	0	0	0	5.00	3.75	0	0	28.75	0
B_	1.25	0	0	2.50	5.00	0	0	8.75	0	0	13.75
C_	13.75	0	0	11.25	65.00	6.25	0	0	3.75	0	0
D_	0	0	0	95.00	1.25	0	0	0	0	0	0
E_	0	0	0	0	93.75	0	0	0	0	0	0
F_	0	0	0	0	0	92.50	0	6.25	0	0	0
G_	8.75	0	0	0	41.25	0	0	33.75	1.25	0	0
H_	0	0	0	0	0	0.	0	53.75	7.50	0	3.75
I_	0	0	0	12.50	0	0	0	25.00	3.75	0	0
J_	0	0	0	0	1.25	3.75	0	15.00	30.00	0	3.75
K_	0	0	0	0	2.50	3.75	0	16.25	0	0	52.50
L_	1.25	0	0	0	0	0	0	0	0	0	1.25
M_	55	0	0	0	2.50	0	0	2.50	16.25	0	3.75
N_	0	0	0	0	1.25	6.25	0	3.75	0	0	32.50
O_	0	0	0	38.75	0	0	0	10	22.50	0	0
P_	0	0	0	0	0	0	0	3.75	0	0	0
R_	1.25	0	0	1.25	3.75	0	0	5	0	0	48.75
S_	0	0	0	0	6.25	1.25	0	13.75	0	0	0
T_	0	0	0	0	0	51.25	0	0	0	0	1.25
U_	0	0	0	0	0	0	0	20	32.50	0	1.25
W_	2.50	0	0	0	2.50	18.75	0	0	6.25	0	50
X_	0	0	0	7.50	0	5	0	3.75	0	0	25
Y_	0	0	0	0	3.70	60.49	0	25.93	0	0	3.70
Z_	0	0	0	5	20	0	0	0	0	0	11.25

Litera->	L	M	N	O	P	R	S	T	U	W	X	Y	Z
A_	0	0	0	42.50	0	0	0	0	0	0	3.75	0	0
B_	1.25	0	0	0	0	0	15	3.75	0	0	0	0	48.75
C_	0	0	0	0	0	0	13.75	0	0	0	0	0	0
D_	1.25	0	0	0	2.50	0	0	0	0	0	0	0	0
E_	0	0	0	0	3.75	0	1.25	0	0	0	0	0	1.25
F_	0	0	0	0	0	0	0	1.25	0	0	0	0	0
G_	0	0	0	0	1.25	0	6.25	0	0	0	0	0	7.50
H_	0	0	0	0	0	0	0	2.50	0	0	32.50	0	0
I_	0	0	0	0	0	0	0	58.75	0	0	0	0	0
J_	0	0	0	0	7.50	0	1.25	28.75	0	0	0	0	8.75
K_	0	0	0	0	2.50	0	0	3.75	0	0	16.25	0	2.50
L_	97.50	0	0	0	0	0	0	0	0	0	0	0	0
M_	2.50	0	0	0	0	0	0	0	0	0	13.75	0	3.75
N_	0	0	0	0	8.75	0	0	0	0	0	45	0	2.50
O_	0	0	0	0	17.50	0	0	11.25	0	0	0	0	0
P_	0	0	0	0	96.25	0	0	0	0	0	0	0	0
R_	2.50	0	0	0	6.25	0	0	0	0	0	2.50	0	28.75
S_	0	0	0	0	0	0	77.50	1.25	0	0	0	0	0
T_	0	0	0	0	0	0	0	47.50	0	0	0	0	0
U_	0	0	0	0	1.25	0	2.50	0	0	0	0	0	42.50
W_	0	0	0	0	1.25	0	0	5	0	0	13.75	0	0
X_	1.25	0	0	0	0	0	3.75	12.50	0	0	40	0	1.25
Y_	0	0	0	0	0	0	0	0	0	0	0	0	6.17
Z_	6.25	0	0	0	1.25	0	28.75	6.25	0	0	2.50	0	18.75

Testowana sieć dla małych liter.

- Minimalna sprawność sieci 30
- $no = 8$
- $m0 = 5$
- Liczba neuronów warstwy ukrytej 17
- Zbiór uczący male_train (80 obrazków dla każdej litery)
- Zbiór testowy male_test (5 obrazków dla każdej litery)

Sprawność sieci 39.49%.

W poniższej tabeli kolejne wiersze, to kolejne małe litery, a kolumny mówią ile procent odpowiadzi było na jaką literę.

Litera->	m	n	o	p	r	s	t	u	w	x	y	z
mala_literka_a_	0	60	0	0	0	0	0	0	40	0	0	0
mala_literka_b_	0	0	0	0	0	0	0	0	20	0	0	0
mala_literka_c_	0	0	0	0	0	0	0	0	0	0	0	0
mala_literka_d_	0	20	0	0	0	0	0	0	80	0	0	0
mala_literka_e_	0	0	0	0	0	0	0	0	0	0	0	0
mala_literka_f_	0	40	0	0	0	0	0	0	20	0	0	0
mala_literka_g_	0	40	0	0	0	0	0	0	0	0	0	0
mala_literka_h_	0	0	0	0	0	0	0	0	80	0	0	0
mala_literka_i_	0	40	0	0	0	20	0	0	0	0	0	0
mala_literka_j_	0	20	0	0	0	0	0	0	0	0	0	0
mala_literka_k_	0	0	0	0	0	0	0	0	60	0	0	0
mala_literka_l_	0	0	0	0	0	0	0	0	40	0	0	0
mala_literka_m_	0	100	0	0	0	0	0	0	0	0	0	0
mala_literka_n_	0	100	0	0	0	0	0	0	0	0	0	0
mala_literka_o_	0	0	0	0	0	40	0	0	20	0	0	0
mala_literka_p_	0	40	0	0	0	0	0	0	20	0	0	0
mala_literka_r_	0	40	0	0	0	0	0	0	60	0	0	0
mala_literka_s_	0	0	0	0	0	100	0	0	0	0	0	0
mala_literka_t_	0	20	0	0	0	60	0	0	0	0	0	0
mala_literka_u_	0	40	0	0	0	0	0	0	20	0	0	0
mala_literka_w_	0	0	0	0	0	0	0	0	80	0	0	0
mala_literka_x_	0	80	0	0	0	20	0	0	0	0	0	0
mala_literka_y_	0	80	0	0	0	0	0	0	0	0	0	0
mala_literka_z_	0	100	0	0	0	0	0	0	0	0	0	0

Testowana sieć dla znaków.

- Minimalna sprawność sieci 90
- $no = 8$
- $m0 = 5$
- Liczba neuronów warstwy ukrytej 10
- Zbiór uczący male_train (50 obrazków dla każdej cyfry)
- Zbiór testowy male_test (30 obrazków dla każdej cyfry)

Sprawność sieci 100%.

W poniższej tabeli kolejne wiersze, to kolejne znaki, a kolumny mówią ile procent odpowiedzi było na jaki znak.

Znak->	>	<	=
gt	100	0	0
lt	0	100	0
eq	0	0	100

WNIOSKI:

- Bardzo mała rozpoznawalność stosunkowo prostej sieci (dla małych i wielkich liter).
- Za mało obserwacji w zbiorze uczącym.
- Należy zmienić parametry sieci.

6.5. Opis zmian w stosunku do poprzednio dostarczonych dokumentów

Zrezygnowałam z wydziałania kilku znaków niepołączonych ze sobą, gdyż o wiele lepsze efekty uzyskuje się zostawiając puste miejsca w działaniach matematycznych.

Niestety, pojawiły się przewidywane (w analizie technicznej) problemy - sieć bardzo dobrze radzi sobie z cyframi, jednak znacznie gorzej z alfabetem (małe litery, a najgorzej z dużymi literami).

Źródłem ów problemów może być:

- Źle skonfigurowana, za mało uczona sieć neuronowa (zbyt mała rozdzielczość wprowadzanego obrazu, zbyt mało lub zbyt dużo neuronów w warstwie wejściowej i ukrytej, zbyt krótki czas uczenia się sieci - za mało epok).
- Niewystarczająco przerobione obrazy (obraz niespójny, dużo szumów, zbyt cienka lub za gruba linia pisma).

Podsumowanie

Podsumowując działanie stworzonego projektu e-podręcznika do nauczania podstawowego, mogę stwierdzić, iż system wykazuje zadowalające działanie dla cyfr oraz znaków, czyli dla alfabetów z niewielką ilością elementów (do 10).

Główną trudnością w danym projekcie było zespojenie 3 modułów (przetwarzanie obrazu, sztuczne sieci neuronowe oraz graficznego interfejsu użytkownika). Celem tworzonego systemu było jak najlepsze rozpoznawanie obrazu. Niestety, stworzona sieć neuronowa bardzo dobrze działa dla cyfr oraz znaków, jednak dużo gorzej radzi sobie z rozpoznawaniem symboli alfabetu (liter, których jest znacznie więcej). Powodów ku temu może być wiele - zaczynając od procesu przetwarzania obrazu, a kończąc na architekturze i sposobie oraz czasie uczenia (trenowania) sieci neuronowej.

Aby uzyskać lepsze wyniki w rozpoznawaniu liter, można rozbudować moduł przetwarzania obrazu, a także zmienić parametry sieci neuronowej. Do czynności mogących usprawnić sieć neuronową należą:

- Zwiększenie rozdzielczości wprowadzanego obrazu (a tym samym zwiększenie ilości neuronów w warstwie wejściowej sieci neuronowej).
- Dokładniejsza obróbka wstępna obrazu (eliminacja szumów, tworzenie szkieletu symboli).
- Dodanie neuronów w warstwie ukrytej sieci neuronowej.
- Zmiana parametrów uczenia sieci neuronowej.
- Więcej przykładów symboli w zbiorze uczącym.
- Większa różnorodność symboli w zbiorze uczącym (możliwe, iż użyte symbole są do siebie zbyt zbliżone).

Otrzymana wersja e-podręcznika nadaje się więc do nauki pisania cyfr i znaków, a także liczenia. Aby dziecko mogło w pełni korzystać z projektu, należy dopracować sieć neuronową odpowiadającą za rozpoznawanie liter, aby otrzymana sprawność sieci wynosiła co najmniej 85%.

Podsumowując, mogę zgodzić się z Edwardem Chylińskim w [11], iż trudne i pracochłonne projektowanie oraz uczenie sztucznych sieci neuronowych należy uznać, co najmniej, w takim samym stopniu za sztukę, jak i naukę. Nauka bowiem stworzyła teorię - biologiczne podstawy budowy oraz działania sieci i związane z nimi podstawy matematyczne. Sztuką natomiast jest umiejętność dokonania właściwego wyboru oraz przygotowania danych do wygenerowania ciągu uczącego i modelowanie współczynników mających wpływ na efekt końcowy (η, β, α).

Kolejnym krokiem do wzrostu efektywności sieci neuronowych jest połączenie możliwości predykcji związanych z sieciami neuronowymi, z przygotowaniem dla nich ciągów uczących za pomocą algorytmów genetycznych. Wynikiem takiego działania byłoby zwielokronienie możliwości w przetwarzaniu danych.

Następnym etapem w rozwoju przedstawionego przeze mnie projektu jest stworzenie systemu OCR (ang. *Optical Character Recognition*) rozpoznającego całe wyrazy lub nawet zdania i akapity.

Spis rysunków

1.1. Ludzki mózg potrafi przewidzieć kolejność liter.	11
2.1. Transformacje pomiędzy klasami. (Na podstawie [8], s.62)	14
2.2. Etapy rozpoznawania obrazów I.	15
2.3. Etapy rozpoznawania obrazów II.	15
2.4. Obraz rzeczywisty oraz obraz zarejestrowany	16
2.5. Etapy rozpoznawania obrazów III.	17
3.1. Budowa neuronu. (Na podstawie [9] 2.1)	22
3.2. Schemat budowy sztucznego neuronu na podstawie neuronu biologicznego. (Na podstawie [9] 2.2)	22
3.3. Schemat budowy sztucznego neuronu. (Na podstawie [1] 6.2.1, s.168)	23
3.4. Funkcje aktywacji	25
3.5. Nieliniowe funkcje aktywacji i ich pochodne. (Na podstawie [11] Tabela 1)	26
3.6. Model perceptronu. (Na podstawie [1] 6.2.2., s.169)	27
3.7. Model adeline. (Na podstawie [1] 6.2.3., s.175)	28
3.8. Model neuronu sigmoidalnego. (Na podstawie [1] 6.2.4., s.180)	28
3.9. Model neuronu Hebba. (Na podstawie [1] 6.2.5., s.184)	29
3.10. Schemat budowy jednokierunkowej, nieliniowej sztucznej sieci neuronowej.	30
3.11. Uproszczony schemat budowy wielowarstwowej sztucznej sieci neuronowej. (Na podstawie [7] 4.2., s.5)	31
3.12. Schemat przebiegu algorytmu wstecznej propagacji błędów. (Na podstawie [12] 1.6.1.)	32
3.13. Klasyfikacja sieci neuronowych. (Na podstawie [12] 1.4.)	35
4.1. Przypadki użycia	38
4.2. Budowa sieci <i>cyfry</i>	42
4.3. Pomyślnie utworzona sieć	43
4.4. Działanie dodawania	45
4.5. Porównywanie działań	45
4.6. Uzupełnianie małych liter	46
4.7. Pisanie cyfr	46
4.8. Pisanie małych liter	47
4.9. Pisanie wielkich liter	47
4.10. Wprowadzanie wyniku	48
4.11. Uzupełnianie wielkich liter	48
4.12. Błąd przy wprowadzaniu symbolu	49
4.13. Dobry wynik, dobry obraz	49
4.14. Niepoprawny wynik, cyfry się nie zgadzają	50

4.15. Uzupełnianie małych liter	50
4.16. Dobra odpowiedź, niedokładnie napisana	51
4.17. Podsumowanie	51
4.18. Diagram komponentów	52
4.19. Struktura systemu OCR	53
4.20. Wczytana literka A oraz po przekształceniu.	54
4.21. Sieć neuronowa w trakcie nauki	56
4.22. Sieć neuronowa użyta w programie do rozpoznawania cyfr.	56
5.1. Główna aplikacja do budowania oraz trenowania sieci.	60
5.2. Główna aplikacja do rozpoznawania symboli	60
5.3. Okno programu.	61
5.4. Główne okno programu.	64

Bibliografia

- [1] L. Rutkowski: *Metody i techniki sztucznej inteligencji*, Wydawnictwo Naukowe PWN, Warszawa 2005
- [2] R. Ptak: *Metody analizy wybranych cech pisma ręcznego na potrzeby inteligentnego systemu obliczeniowego (rozprawa doktorska)*, Instytut Informatyki, Automatyki i Robotyki Politechniki Wrocławskiej, Wrocław 2006
- [3] S. Impedovo: *FROM PIXELS TO FEATURES III Frontiers in Handwriting Recognition*, ELSEVIER SCIENCE PUBLISHER B.V., Amsterdam 1992
- [4] B. Cyganek: *Komputerowe przetwarzanie obrazów trójwymiarowych*, EXIT, Warszawa 2002
- [5] R. Tadeusiewicz : *Komputerowa analiza i przetwarzanie obrazów.*, Wydawnictwo Fundacji Postępu Telekomunikacji, Kraków 1997
- [6] M. Ostrowski (red): *Informacja obrazowa*, WNT, Warszawa 1992
- [7] S. Teresi: *A study of Handwriting Recognition*, Urbana-Champaign: University of Illinois, 1998
Dostępny w internecie: <http://teresi.us/html/writing/handwriting.html>
- [8] K. Krawiec: *Rozpoznawanie i Przetwarzanie Obrazów* , Poznań 2010
Dostępny w internecie: <http://www.cs.put.poznan.pl/kkrawiec/wiki/uploads/RO/po1.pdf>
- [9] Ch. Stergiou, D. Siganos: *NEURAL NETWORKS*
Dostępny w internecie: http://www.doc.ic.ac.uk/~nd/surprise_96/journal/vol4/cs11/report.html
- [10] W. Kacalak: *NEW METHODS FOR HANDWRITING RECOGNITION USING ARTIFICIAL NEURAL NETWORKS*
Technical University of Koszalin, Koszalin 2006
- [11] E. Chyliński: *Sztuczne sieci neuronowe rozpoznawanie wzorców - podejście praktyczne*
Dostępny w internecie: http://edward_ch.republika.pl/pod_prakt.html
- [12] *Sztuczne sieci neuronowe i algorytmy genetyczne*
Dostępny w internecie:
http://4programmers.net/Z_pogranicza/Sztuczne_sieci_neuronowe_i_algorytmy_genetyczne#r1

Marta Jankowska
Nr albumu 248965

Warszawa, 24 maja 2012

Oświadczenie

Oświadczam, że pracę inżynierską pod tytułem „Projekt e-podręcznika do nauczania podstawowego E-learning Project For Primary Level Education”, której promotorem jest dr inż. Marcin Luckner wykonałam samodzielnie, co poświadczam własnoręcznym podpisem.

.....

Marta Jankowska